



**ŠKOLA ZA DIZAJN, GRAFIKU I ODRŽIVU
GRADNJU**

Matice hrvatske 11, Split

ZAVRŠNI STRUČNI RAD:

**Botanic Journal –
Razvoj sustava za
upravljanje uzgojem
biljaka i vođenje
dnevnika**

UČENIK: Mario Radman

MENTOR: Ana Vidović

RAZRED: 4gP web programer

U Splitu, svibanj 2026.

Sadržaj

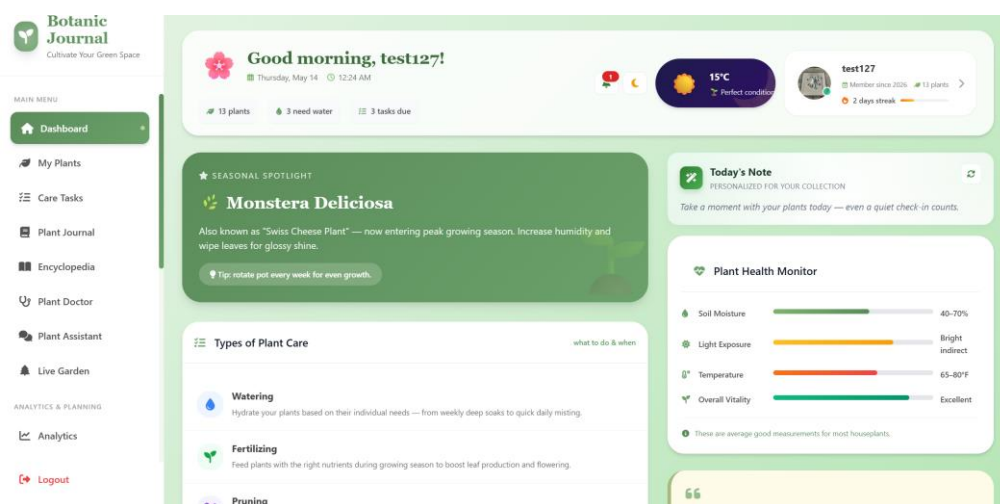
Sadržaj	3
UVOD.....	5
Predstavljanje teme.....	5
Ciljevi i svrha rada.....	6
Metodologija	6
Struktura rada	6
1. Teorijska osnova.....	7
1.1. React.js - Frontend biblioteka	7
1.2. PHP - Backend tehnologija	7
1.3. MySQL - Relacijska baza podataka.....	8
1.4. REST API Arhitektura	8
1.5. Vite - Alat za izgradnju projekta	9
2. Arhitektura sustava	10
2.1. Struktura projekta	10
2.2. Tok podataka (Data Flow).....	10
2.3. Dizajn baze podataka	11
2.4. API servis (api.js)	12
3. Frontend implementacija	14
3.1. Upravljanje stanjem (State Management).....	14
3.2. Komponente i struktura.....	15
3.3. Stilizacija.....	18
3.4. Header s vremenskom prognozom	19
3.5. Kalendar zadataka (Garden Planner)	19

3.6. Analitika s Chart.js	21
3.7. Plant Doctor — AI dijagnostika biljaka..	22
3.8. Diagnosis History — povijest dijagnoza	24
3.9. Garden Studio — Garden Map Designer	25
3.10. Garden Studio — AI Zone Fit analiza ..	26
3.11. Garden Studio — Live Garden panorama	27
3.12. Today's Note — personalizirana dnevna poruka	28
3.13. Direct Messages — privatne poruke.....	29
4. Backend implementacija	30
4.1. Autentikacija (auth.php).....	30
4.2. Upravljanje biljkama (plants.php)	31
4.3. Community forum (community/)	32
4.4. Plant Requests sustav	35
4.5. Admin panel	36
4.6. Upload datoteka	38
4.7. Plant Doctor API (plant-doctor.php)	38
4.8. Plant Assistant API (plant-chat.php)	39
4.9. Garden Map placements (garden- map.php)	40
4.10. AI Zone Fit (garden-map-tip.php)	41
4.11. Today's Note (daily-care-note.php)	42
4.12. Direct Messages (direct-messages.php)	42
4.13. Social — follow/unfollow (social.php).	43
5. Sigurnosni aspekti	45
5.1. Zaštita lozinki	45

5.2. SQL Injection zaštita.....	46
5.3. CORS politika	46
5.4. Autorizacija	47
ZAKLJUČAK	48
LITERATURA.....	49

UVOD

Botanic Journal je full-stack web aplikacija koja korisnicima omogućuje vođenje digitalnog dnevnika biljaka, postavljanje podsjetnika za zalijevanje i gnojidbu, pregledavanje enciklopedije biljaka te sudjelovanje u community forumu (Forumu zajednice) s drugim ljubiteljima biljaka. Sustav je razvijen koristeći React na frontendu i PHP/MySQL na backendu.



Slika 1: Botanic Journal - Dashboard sučelje

Predstavljanje teme

Tema ovog rada je razvoj moderne, responzivne web aplikacije za praćenje i njegu biljaka. Aplikacija obuhvaća čitav ekosustav funkcionalnosti: od osobne kolekcije biljaka i zadataka njege, do zajedničkih foruma i enciklopedije biljaka.

Briga o biljkama zahtijeva redovitost, znanje i organizaciju. Mnogi ljubitelji biljaka gube biljke zbog neodgovarajuće njege, preskočenih zalijevanja ili neprepoznatih bolesti. Digitalni alat koji bi ih podsjećao na zadatke njege i pružao informacije o svakoj biljci može znatno poboljšati njihov uspjeh.

Ciljevi i svrha rada

- Razviti funkcionalan cjelovit web sustav za praćenje biljaka
- Implementirati sustav autentikacije i autorizacije korisnika
- Izraditi enciklopediju biljaka s mogućnošću dodavanja u osobnu kolekciju
- Razviti community forum (Forum zajednice) za razmjenu znanja između korisnika
- Omogućiti administraciju sadržaja putem admin panela
- Primijeniti responzivni dizajn koji radi na svim uređajima

Metodologija

U razvoju aplikacije korišten je komponentni pristup (component-based architecture) uz React biblioteku za frontend. Backend je implementiran kao REST API u PHP-u, a podaci se pohranjuju u MySQL relacijsku bazu podataka. Razvojni proces slijedio je agilne principe s iterativnim razvojem funkcionalnosti.

Struktura rada

Rad je organiziran u šest glavnih poglavlja. U prvom poglavlju opisana je teorijska osnova web tehnologija korištenih u projektu. Drugo poglavlje posvećeno je arhitekturi sustava i dizajnu baze podataka. Treće poglavlje opisuje frontend implementaciju u Reactu, uključujući napredne module poput Plant Doctora, Garden Studija i AI Plant Assistanta. Četvrto poglavlje pokriva backend PHP API s detaljnim opisom svih endpointa. Peto poglavlje opisuje sigurnosne aspekte. Zaključak sažima postignuća i rezultate razvoja.

1. Teorijska osnova

Botanic Journal je izgrađen na modernom skupu web tehnologija koje zajedno čine stabilnu, skalabilnu i korisničku aplikaciju. U ovom poglavlju opisane su temeljne tehnologije i koncepti koji su korišteni u razvoju.

1.1. React.js - Frontend biblioteka

React je JavaScript biblioteka za izgradnju korisničkih sučelja, razvijena od strane Meta (Facebook). Temelji se na konceptu komponenti (components), gradivnih blokova sučelja koji upravljaju vlastitim stanjem i izgledom.

Ključni koncepti Reacta korišteni u projektu:

- JSX sintaksa - kombinacija JavaScripta i XML-a koja opisuje strukturu komponente
- useState i useEffect hookovi za upravljanje stanjem i životnim ciklusom komponenti
- Props (svojstva) za prijenos podataka između roditeljskih i dječjih komponenti
- React Router DOM za navigaciju između stranica bez ponovnog učitavanja
- Virtualni DOM za učinkovito ažuriranje sučelja

Aplikacija koristi React 19.2.0, najnoviju stabilnu verziju, zajedno s Vite 7.2.2 kao alatom za izgradnju (build tool) koji pruža izuzetno brzo razvojno okruženje.

1.2. PHP - Backend tehnologija

PHP (Hypertext Preprocessor) je serverski skriptni jezik namijenjen web razvoju. U ovom projektu PHP se koristi za izgradnju REST API-ja koji prima HTTP zahtjeve od React frontenda, komunicira s MySQL bazom podataka i vraća JSON odgovore.

PHP backend organiziran je u sljedeće slojeve:

- API sloj (/backend/api/) - direktna obrada HTTP zahtjeva
- Model sloj (/backend/models/) - klase koje predstavljaju entitete baze podataka
- Konfiguracija (/backend/config/) - podešavanja baze podataka i CORS

Svaki API endpoint (plants.php, tasks.php, journals.php, auth.php itd.) rukuje različitim HTTP metodama (GET, POST, PUT, PATCH, DELETE) i vraća standardizirane JSON odgovore s poljem 'success' i podacima.

1.3. MySQL - Relacijska baza podataka

MySQL je open-source sustav za upravljanje relacijskim bazama podataka (RDBMS). Podaci se pohranjuju u tablice s unaprijed definiranom strukturom, a veze između tablica uspostavljaju se putem stranih ključeva (foreign keys).

1.4. REST API Arhitektura

REST (Representational State Transfer) je arhitekturni stil za izgradnju web usluga. REST API koristi standardne HTTP metode za operacije nad resursima:

- GET - dohvaćanje podataka (npr. GET /api/plants.php?user_id=5)
- POST - kreiranje novog resursa (npr. POST /api/plants.php)
- PUT - potpuna izmjena resursa (npr. PUT /api/journals.php?id=3)
- PATCH - djelomična izmjena resursa (npr. PATCH /api/plants.php za toggle favorit)
- DELETE - brisanje resursa (npr. DELETE /api/tasks.php?id=7)

Svi API endpointi vraćaju JSON format s poljem success (true/false), message za opis i data za stvarne podatke. Autentikacija se provodi putem user_id parametra koji se šalje uz svaki zahtjev.

1.5. Vite - Alat za izgradnju projekta

Vite je moderni alat za izgradnju frontend projekata koji koristi native ES module za izuzetno brzo pokretanje razvojnog servera. Za razliku od tradicionalnih alata poput Webpacka, Vite ne bundlira kod u razvojnem načinu rada, što rezultira gotovo trenutnim hot-module-replacement (HMR) ažuriranjima.

2. Arhitektura sustava

Botanic Journal prati client-server arhitekturu s jasnom podjelom odgovornosti između frontenda i backenda. Ovo poglavlje opisuje strukturu projekta, tok podataka i dizajn baze podataka.

2.1. Struktura projekta

Projekt je organiziran u dvije glavne mape:

Mapa	Opis
/src	React frontend izvorni kod
/src/components	React komponente organizirane po funkcionalnosti (Admin, Auth, Community, Dashboard, Encyclopedia, Journal, Layout, Plants, Tasks, Profile...)
/src/services	api.js - centralna klasa za sve HTTP pozive prema backendu
/backend/api	PHP API endpointi (plants.php, tasks.php, journals.php, auth.php, community/...)
/backend/models	PHP model klase (Plant.php, Journal.php, Task.php, CommunityDiscussion.php...)
/backend/config	database.php (PDO konekcija), cors.php (CORS zaglavlja)
/backend/uploads	Uploadane slike (avatari, biljke, zahtjevi za biljke)

2.2. Tok podataka (Data Flow)

Komunikacija između frontenda i backenda odvija se na sljedeći način:

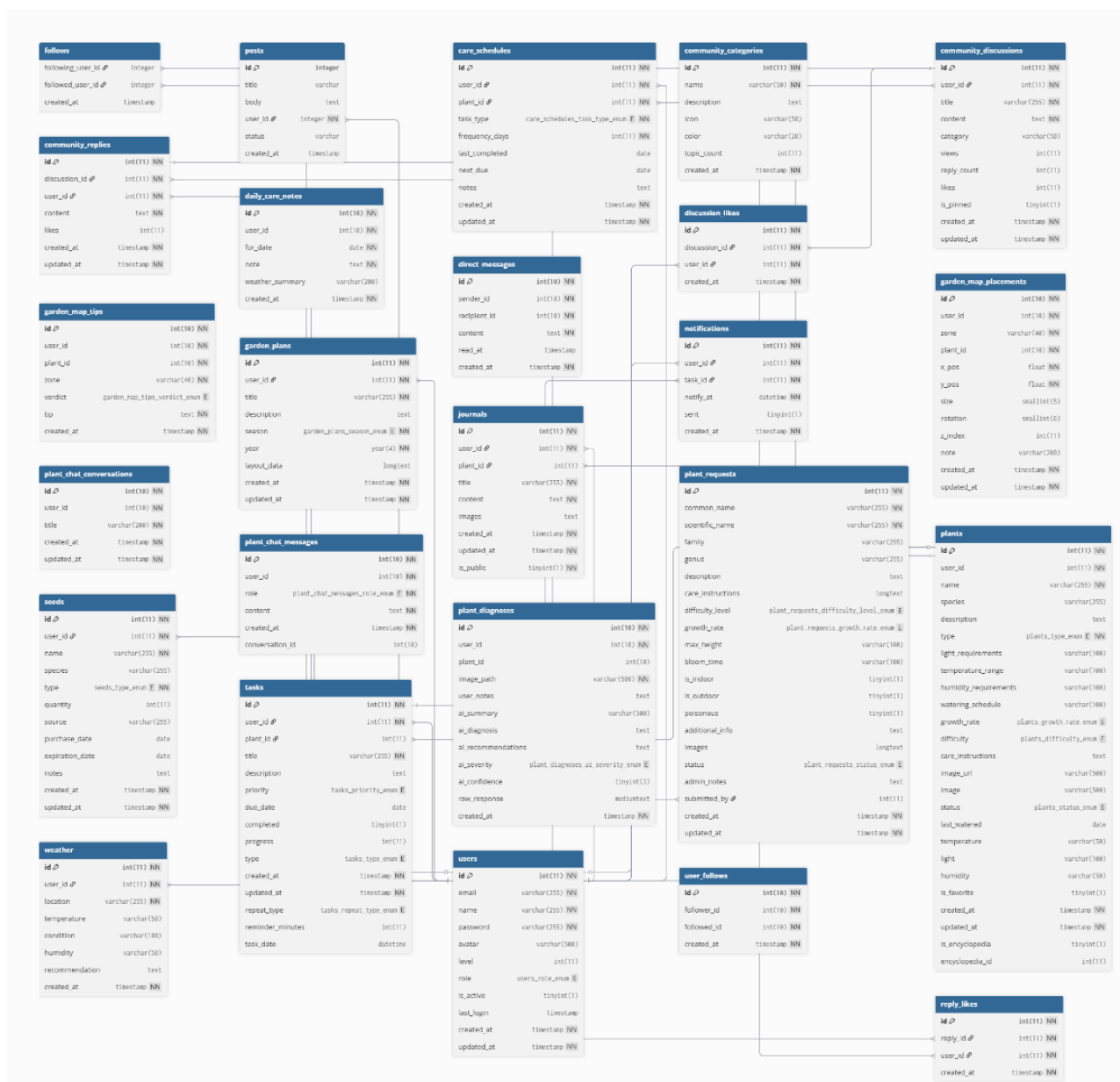
- Korisnik pokreće događaj (event) koji je definiran unutar React komponente (npr. odabire opciju „Dodaj biljku”)
- Komponenta poziva odgovarajuću metodu iz ApiService klase (api.js)
- ApiService formira HTTP zahtjev koristeći fetch() funkciju, uključujući odgovarajući URL, HTTP metodu i tijelo zahtjeva
- PHP backend prima zahtjev, provjerava autentikaciju i komunicira s MySQL

- PHP vraća odgovor u JSON formatu koji sadrži polja success (true/false) i data s pripadajućim podacima
- Komponenta ažurira lokalno stanje (useState) i prikazuje promjene korisniku
- Svi zahtjevi sadržavaju user_id parametar koji se dohvaća iz localStorage. Ovaj pristup osigurava da svaki korisnik vidi i upravlja samo svojim podacima.

2.3. Dizajn baze podataka

Baza podataka temelji se na principima referencijalnog integriteta, koji se ostvaruju primjenom stranih ključeva. Središnja tablica je users, dok sve ostale tablice uspostavljaju vezu s njom putem atributa user_id, čime se osigurava dosljednost i povezanost podataka u sustavu. Tablica plants je posebno zanimljiva jer služi i kao korisnička kolekcija (is_encyclopedia = 0) i kao enciklopedija (is_encyclopedia = 1), a veza encyclopedia_id pamti iz koje enciklopedijske biljke je kopija nastala.

Svako brisanje korisnika kaskadno briše sve njegove podatke (DELETE CASCADE) čime se osigurava konzistentnost podataka u bazi. Tablice community_discussions i community_replies čuvaju like-ove u zasebnim tablicama discussion_likes i reply_likes kako bi se spriječilo duplo lajkanje.



Slika 2: ER dijagram baze podataka

2.4. API servis (api.js)

Centralna ApiService klasa u /src/services/api.js apstrahira svu komunikaciju s backendom. Klasa sadrži više od 60 metoda organiziranih po domenama: autentikacija, biljke, zadaci, dnevnik, enciklopedija, forum zajednice, administracijske funkcionalnosti te sustav prijedloga novih biljaka.

Metoda `getCurrentUserId()` dohvaća ID korisnika iz `localStorage` i automatski ga dodaje svim zahtjevima. Metoda `request()` je generička metoda koja koristi `Fetch API`, parsira JSON odgovor i baca grešku ako zahtjev nije uspio. Za razliku od većine API metoda koje koriste JSON format za razmjenu podataka, metode `uploadAvatar()` i `createAdminPlant()` koriste objekt `FormData` kako bi omogućile prijenos (slikovnih) datoteka i ostalih podataka obrasca.

Backend sadrži 22 API datoteke organizirane po domenama: `auth.php`, `plants.php`, `plants-encyclopedia.php`, `tasks.php`, `journals.php`, `weather.php`, `analytics.php`, `profile.php`, `update-avatar.php`, `user.php`, `user-stats.php`, `user-dashboard.php`, `user-notifications.php`, `stats.php`, `community/` (5 endpointa), `plant-requests.php`, `plant-doctor.php`, `plant-chat.php`, `garden-map.php`, `garden-map-tip.php`, `daily-care-note.php`, `direct-messages.php`, `social.php` i `admin/` (3 endpointa).

```
src > services > JS api.js > ApiService > toggleLockDiscussion
3  class ApiService {
24  async request(endpoint, options = {}) {
62  }
63
64  // Plants
65  async getPlants() {
66      const user_id = this.getCurrentUserId();
67      return this.request('plants.php?user_id=${user_id}');
68  }
69
70  async createPlant(plantData) {
71      const user_id = this.getCurrentUserId();
72      return this.request('plants.php', {
73          method: 'POST',
74          body: { ...plantData, user_id }
75      });
76  }
77
78  async updatePlant(id, plantData) {
79      const user_id = this.getCurrentUserId();
80      return this.request('plants.php?id=${id}', {
81          method: 'PUT',
82          body: { ...plantData, user_id }
83      });
84  }
85
86  async deletePlant(id) {
87      const user_id = this.getCurrentUserId();
88      return this.request('plants.php?id=${id}&user_id=${user_id}', {
89          method: 'DELETE'
90      });
91  }
```

Slika 3: `api.js` - `ApiService` klasa u VS Codeu

3. Frontend implementacija

Navigacija između stranica odvija se putem hash-based routing mehanizma implementiranog direktno u App.jsx, bez potrebe za posebnim routing bibliotekama poput React Router na razini cijele aplikacije.

3.1. Upravljanje stanjem (State Management)

Aplikacija koristi React-ov ugrađeni useState hook za lokalno stanje komponenti i props za prijenos podataka prema dole. Globalno stanje (prijavljeni korisnik, autentikacija) čuva se u App.jsx i prosljeđuje kao props podkomponentama.

Korisnikovi podaci pohranjuju se u localStorage (isAuthenticated, user, user_id) i inicijaliziraju se pri pokretanju aplikacije. Ovo omogućuje perzistenciju sesije između osvježavanja stranice.

Globalno stanje aplikacije pohranjeno je u korijenskoj komponenti App.jsx putem useState hookova: activeView, notifications, isAuthenticated, authView, currentUser, selectedPlantId, showPlantDetailModal, selectedDiscussionId, selectedProfileId i chatWithUserId. Inicijalne vrijednosti isAuthenticated i currentUser čitaju se iz localStorage putem lazy inicializera kako bi sesija perzistirala između osvježavanja stranice.

```
function App() {
  const [activeView, setActiveView] = useState('dashboard');
  const [notifications, setNotifications] = useState([]);
  const [isAuthenticated, setIsAuthenticated] = useState(() => {
    return localStorage.getItem('isAuthenticated') === 'true';
  });
  const [authView, setAuthView] = useState('login');
  const [currentUser, setCurrentUser] = useState(() => {
    const savedUser = localStorage.getItem('user');
    return savedUser ? JSON.parse(savedUser) : null;
  });
  const [selectedPlantId, setSelectedPlantId] = useState(null);
  const [showPlantDetailModal, setShowPlantDetailModal] = useState(false);
  const [selectedDiscussionId, setSelectedDiscussionId] = useState(null);
  const [selectedProfileId, setSelectedProfileId] = useState(null);
  const [chatWithUserId, setChatWithUserId] = useState(null);
}
```

Slika 4: App.jsx — useState hookovi i lazy initializer iz localStorage

3.2. Komponente i struktura

Aplikacija sadrži 30 React komponenti organiziranih u podmape:

Komponenta	Opis funkcionalnosti
App.jsx	Korijen aplikacije — routing, autentikacija, notifikacije
Dashboard.jsx	Naslovna stranica s botaničkim savjetima, Plant Health Monitorom i community pregledom
MyPlants.jsx	Osobna kolekcija biljaka s pretragom, filtriranjem i upravljanjem
PlantEncyclopedia.jsx	Enciklopedija biljaka s dodavanjem u kolekciju i export izvještaja (PDF/Word)
PlantDetail.jsx	Detaljna stranica biljke s tabovima: pregled, njega, galerija, savjeti
CareTasks.jsx	Upravljanje zadacima njega s prioritetima i praćenjem napretka
PlantJournal.jsx	Digitalni dnevnik s brojenjem riječi i upravljanjem unosima
CommunityForum.jsx	Forum s kategorijama, pretragom, statistikama, lajkanjem i Gardeners mrežom
Analytics.jsx	Grafikoni biljaka i zadataka (Chart.js) te Carbon Footprint kalkulator
GardenStudio.jsx	Kombinirani modul: Garden Map Designer, Live Garden animacija i Plant Anatomy pregled
PlantAssistant.jsx	AI chat asistent za savjete o biljkama s pohranom konverzacija
PlantDoctor.jsx	Dijagnostika bolesti i štetnika biljaka s preporukama liječenja
AdminDashboard.jsx	Admin panel za upravljanje korisnicima, biljkama i statistikama
Profile.jsx	Korisnički profil s uređivanjem, uploadom avatara i javnim profilima
Header.jsx	Zaglavlje s vremenskom prognozom, streakom i notifikacijama
Sidebar.jsx	Navigacijska traka sa podrškom za desktop i mobilni drawer
GardenMapDesigner.jsx	Drag-and-drop planer biljaka po zonama (Balcony, Backyard, Greenhouse, Indoor, Rooftop, Other)
LiveGarden.jsx	Pixel-art panorama kolekcije s dan/noć ciklusom i animacijama (pčele, leptiri, kiša)
PlantChat.jsx	AI chat asistent (Plant Assistant) s pohranom konverzacija u plant_chat_messages
CarbonOffset.jsx	Carbon Footprint kalkulator (O2/CO2 po tipu biljke)
PlantAnatomy.jsx	Interaktivni prikaz dijelova biljke s opisima
Gardeners.jsx	Discover modul za pretragu i praćenje korisnika (follow/unfollow)
PublicProfile.jsx	Javni profil korisnika s kolekcijom biljaka i javnim journal unosima
Messages.jsx	Privatni razgovori — popis konverzacija i aktivni chat
PlantRequestForm.jsx	Multi-step forma za predlaganje nove enciklopedijske biljke
MyPlantRequests.jsx	Pregled vlastitih prijedloga i statusa odobrenja

PlantRequestManager.js x	Admin upravljanje prijedlozima (approve/reject)
Notification.jsx	Toast notifikacije (success/error/info) s automatskim sakrivanjem
WeatherForecast.jsx	Detaljna prognoza po danima s preporukama za zalijevanje

Aplikacija koristi vlastiti hash-based routing direktno u App.jsx bez React Router biblioteke. URL hash prati aktivni view (npr. #plants, #discussion/12, #plant/5), što omogućuje dijeljenje direktnih linkova i ispravan rad gumba „Back” u pregledniku. Dva useEffect hooka sinkroniziraju stanje s URL-om: jedan sluša hashchange događaj i parsira hash u aktivni view, drugi ažurira window.location.hash kada se activeView promijeni.

```
useEffect(() => {
  const handleHashChange = () => {
    const hash = window.location.hash.replace('#', '');

    // Handle plant detail routes like #plant/123
    if (hash.startsWith('plant/')) {
      const plantId = hash.split('/')[1];
      setSelectedPlantId(plantId);
      setActiveView('plant-detail');
      return;
    }

    // Handle discussion detail routes
    if (hash.startsWith('discussion/')) {
      const discussionId = hash.split('/')[1];
      setSelectedDiscussionId(discussionId);
      setActiveView('discussion-detail');
      return;
    }
  }
});
```

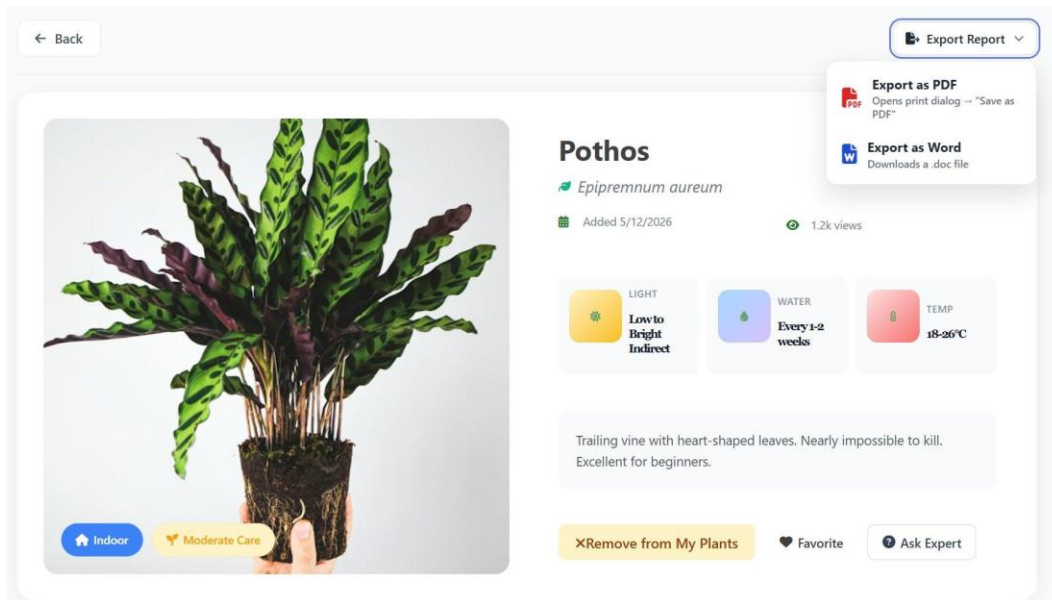
Slika 5: App.jsx — handleHashChange za hash-based routing

Popis svih validnih view-ova definiran je u konstanti `validViews` unutar handlera, što omogućuje sigurnu validaciju primljenih hash vrijednosti prije promjene stanja:

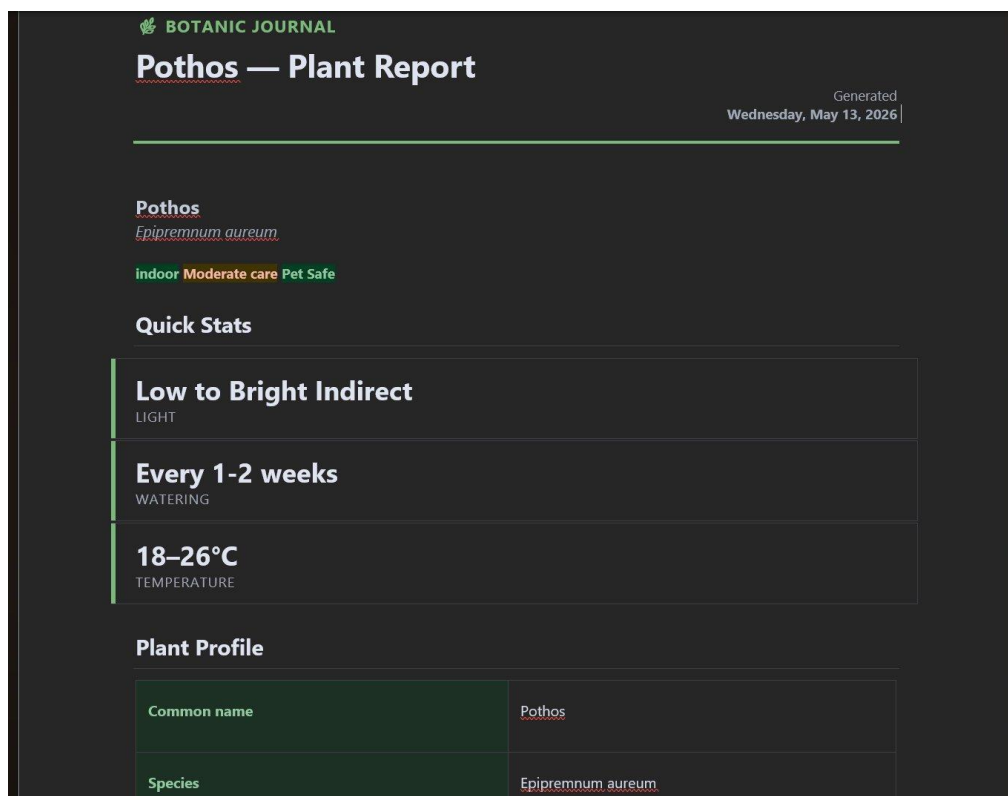
```
const validViews = [  
  'dashboard', 'plants', 'tasks', 'journal', 'encyclopedia',  
  'analytics', 'planner', 'profile', 'plant-detail', 'community',  
  'discussion-detail', 'suggest-plant', 'my-requests', 'plant-requests', 'admin',  
  'plant-doctor', 'plant-chat', 'garden-map',  
  'gardeners', 'messages', 'public-profile',  
  'carbon', 'anatomy', 'live-garden'  
];
```

Slika 6: *App.jsx* — `validViews` lista i validacija primljenih hash vrijednosti

Stranica biljke (*PlantDetail.jsx*) podržava export podataka putem gumba **Export Report**. Dostupan je PDF izvoz (otvara print dialog) i Word format (.doc). Izvještaj prikazuje naziv, vrstu, uvjete njege, sezonske preporuke i Quick Stats biljke.



Slika 7: *Export Report* — opcije izvoza podataka o biljci



Slika 8: Plant Report — generirani PDF izvještaj

3.3. Stilizacija

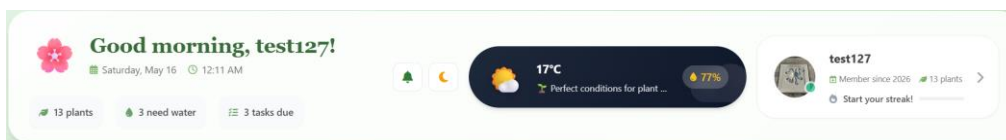
Svaka komponenta ima vlastitu CSS datoteku (App.css, plants.css, tasks.css, community.css itd.) s CSS varijablama definiranim u :root selekciji. Koriste se boje inspirirane botaničkim temama: zelene nijanse (forest-600, sage-500), mint i lavanda naglasci.

Aplikacija sadrži 16 React komponenti organiziranih po funkcionalnosti. Komponente GardenStudio, PlantAssistant i PlantDoctor proširuju aplikaciju AI i vizualnim mogućnostima. Aplikacija je responzivna te koristi CSS media query pravila za prilagodbu prikaza mobilnim uređajima (max-width: 767px). Na mobilnim uređajima desktop bočna navigacija se skriva, dok se umjesto nje prikazuje donja navigacijska traka, uz dodatni drawer izbornik koji se otvara klizanjem od donjeg ruba zaslona prema gore.

3.4. Header s vremenskom prognozom

Header komponenta integrira WeatherAPI.com za prikazivanje stvarnih vremenskih podataka korisnikove lokacije. Koristi navigator.geolocation API za dohvaćanje geografskih koordinata te dinamički mijenja pozadinsku boju weather kartice ovisno o vremenskim uvjetima (sunčano - narančasta, kišno - siva, noć - tamna).

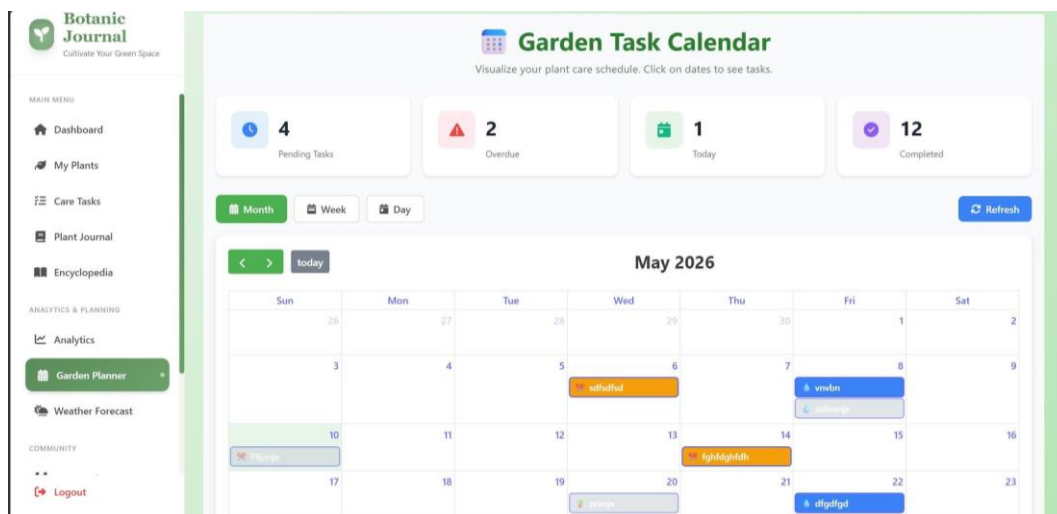
U zaglavlju aplikacije prikazuje se personalizirani pozdrav, vremensku prognozu s preporukom za zalijevanje, brzi pregled statistike biljaka i zadataka, dnevni streak te zvono za notifikacije u realnom vremenu. Notifikacijski sustav prikazuje badge s brojem nepročitanih poruka.



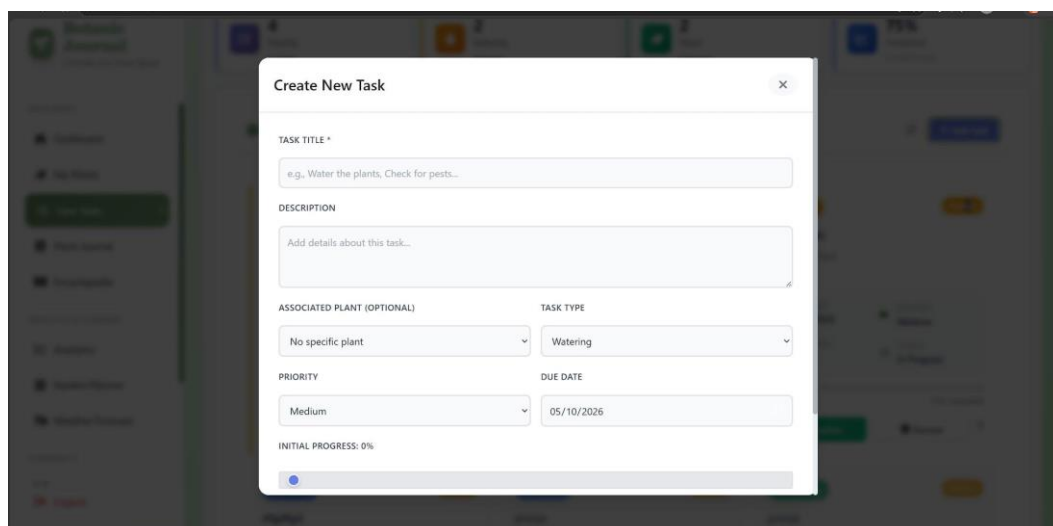
Slika 9: Header s vremenskom prognozom i statistikama

3.5. Kalendar zadataka (Garden Planner)

GardenPlanner komponenta koristi FullCalendar biblioteku za prikaz zadataka njege biljaka u interaktivnom kalendaru. Svaki zadatak je prikazan kao event u odgovarajućoj boji ovisno o tipu (zalijevanje-plava, gnojidba-zelena, rezanje-žuta, repotiranje-ljubičasta, zaštita od štetnika-ružičasta). Klik na event prikazuje detalje zadatka putem showNotification.



Slika 10: Garden Task Calendar - prikaz zadatka na kalendaru

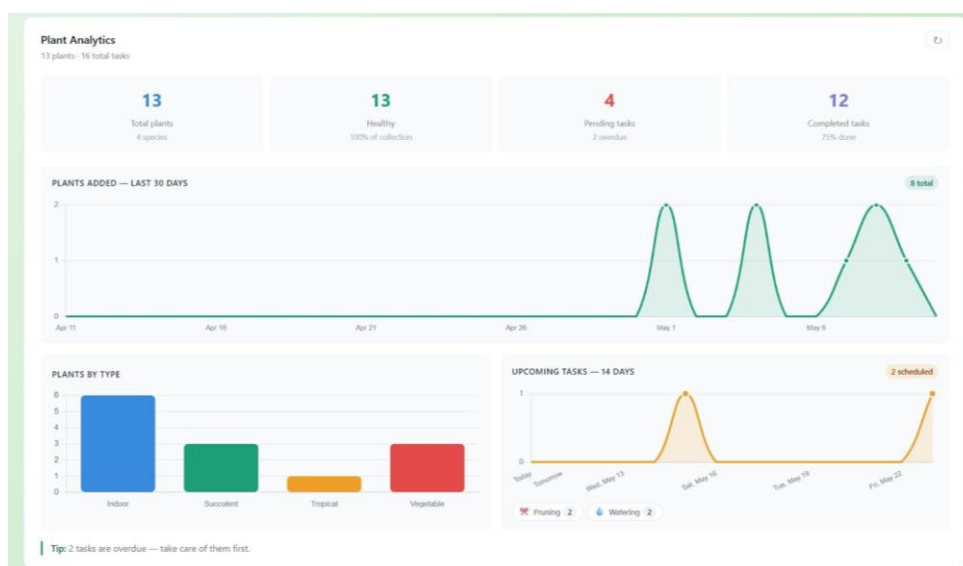


Slika 11: Create New Task - forma za kreiranje zadatka

3.6. Analitika s Chart.js

Analytics komponenta koristi Chart.js (putem react-chartjs-2 biblioteke) za vizualizaciju podataka o biljkama i zadacima. Prikazuje:

- Linijski grafikon dodanih biljaka u zadnjih 30 dana
- Stupčasti grafikon biljaka po kategorijama (indoor, outdoor, succulent, tropical...)
- Linijski grafikon nadolazećih zadataka u sljedećih 14 dana s tooltip pregledom
- Statistične kartice: ukupno biljaka, zdravih biljaka, zaostalih zadataka, stopa završenosti



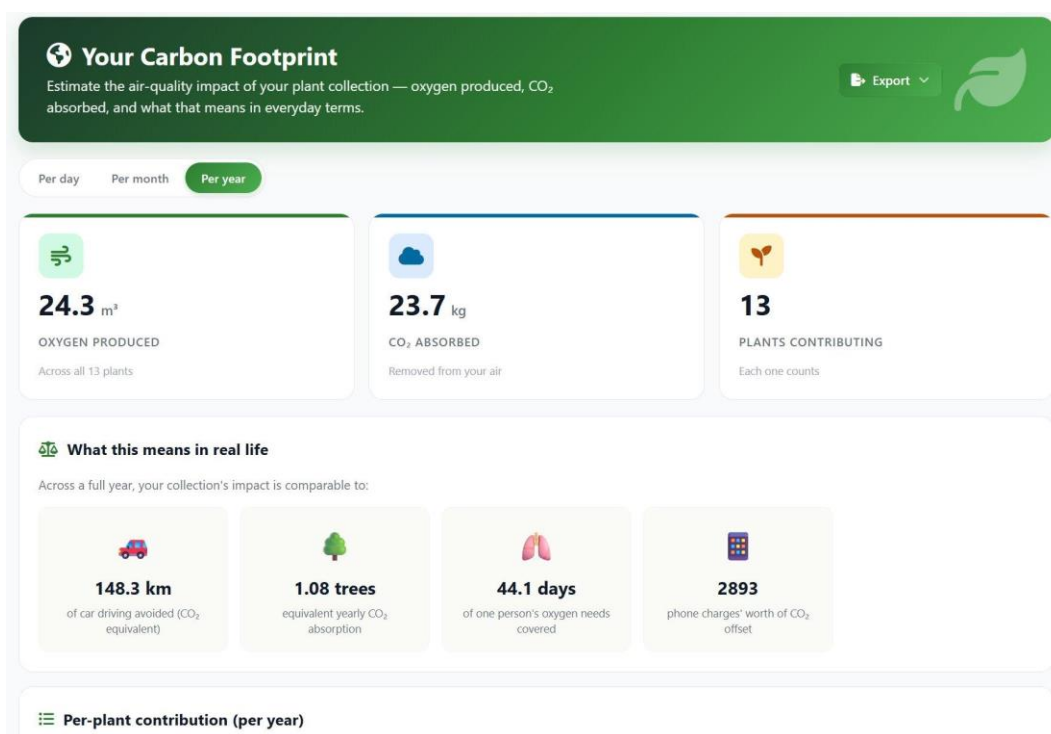
Slika 12: Plant Analytics - grafovi i statistike

Unutar Analytics modula integriran je Carbon Footprint kalkulator koji procjenjuje ekološki učinak kolekcije. Izračunava se količina kisika (m^3) i CO_2 (kg) po danu, mjesecu ili godini, s prikazom u svakodnevnim ekvivalentima: prijedeni km automobila, broj stabala, pokrivenost kisikovih potreba osobe.

```
const COEFFS = {
  succulent: { co2: 1, o2: 1.5 },
  herb: { co2: 4, o2: 4 },
  indoor: { co2: 5, o2: 5 },
  flowering: { co2: 6, o2: 6 },
  vegetable: { co2: 7, o2: 7 },
  tropical: { co2: 8, o2: 8 },
  outdoor: { co2: 15, o2: 15 },
};
```

Slika 13: CarbonOffset.jsx — COEFFS tablica koeficijena po tipu biljke

Ukupna količina dobiva se sumom $\text{COEFFFS}[\text{type}] \times \text{broj_dana}$ za sve biljke u kolekciji. Rezultat se pretvara u svakodnevne ekvivalente: kilometri vožnje automobila ($1 \text{ kg CO}_2 \approx 6.5 \text{ km}$), broj stabala ($1 \text{ stablo} \approx 22 \text{ kg CO}_2/\text{god}$) i pokrivenost dnevnih potreba osobe za kisikom ($\approx 550 \text{ L/dan}$).



Slika 14: Carbon Footprint kalkulator — godišnji ekološki učinak kolekcije

3.7. Plant Doctor — AI dijagnostika biljaka

Plant Doctor je AI-pogonjen modul koji korisnicima omogućuje brzu dijagnostiku stanja biljke pomoću fotografije. Korisnik prenosi sliku (JPG, PNG ili WEBP, do 8 MB), opcionalno upisuje naziv biljke i opis simptoma (npr. „žuti donji listovi, smeđi vrhovi“), a sustav vraća strukturiranu dijagnozu kroz tri sekcije: opis problema, razinu ozbiljnosti (Healthy / Moderate / Severe) i konkretne preporuke za njegu.

```
const handleFile = (file) => {
  if (!file) return;
  if (!/^image\/\w+\.test(file.type)) {
    showNotification('Invalid file', 'Please choose an image (JPG, PNG, WEBP).', 'error');
    return;
  }
  if (file.size > 8 * 1024 * 1024) {
    showNotification('Too large', 'Image must be under 8MB.', 'error');
    return;
  }
  setImageFile(file);
  setImagePreview(URL.createObjectURL(file));
  setLatestResult(null);
};
```

Slika 15: PlantDoctor.jsx — handleFile validacija uploada

Zaglavlje modula prikazuje tri agregatne statistike kolekcije: ukupan broj obavljenih dijagnoza, broj zdravih rezultata i broj biljaka koje zahtijevaju pažnju. Ovaj pregled korisniku služi kao brzi indikator zdravlja cijele kolekcije bez ulaska u pojedinačne zapise.

The screenshot displays the Plant Doctor application interface. At the top, a green header bar contains the app logo and name, a brief description, and three summary statistics: 7 Diagnoses, 3 Healthy Results, and 4 Need Attention. Below the header, the interface is split into two main sections. The left section, titled 'New Diagnosis', features a large dashed box for dropping a photo, with instructions to use JPG, PNG, or WEBP formats up to 8MB. Below this are optional text inputs for 'Which plant?' and 'What are you seeing?' (with an example: 'e.g. Yellow lower leaves, brown crispy tips, started about a week ago...'). A green 'Diagnose with AI' button is at the bottom of this section. The right section, titled 'Diagnosis History', shows a list of past diagnoses. Each entry includes a small image, a text description of the plant's condition, a timestamp, and a status label (HEALTHY, SEVERE, or MODERATE) with a corresponding icon.

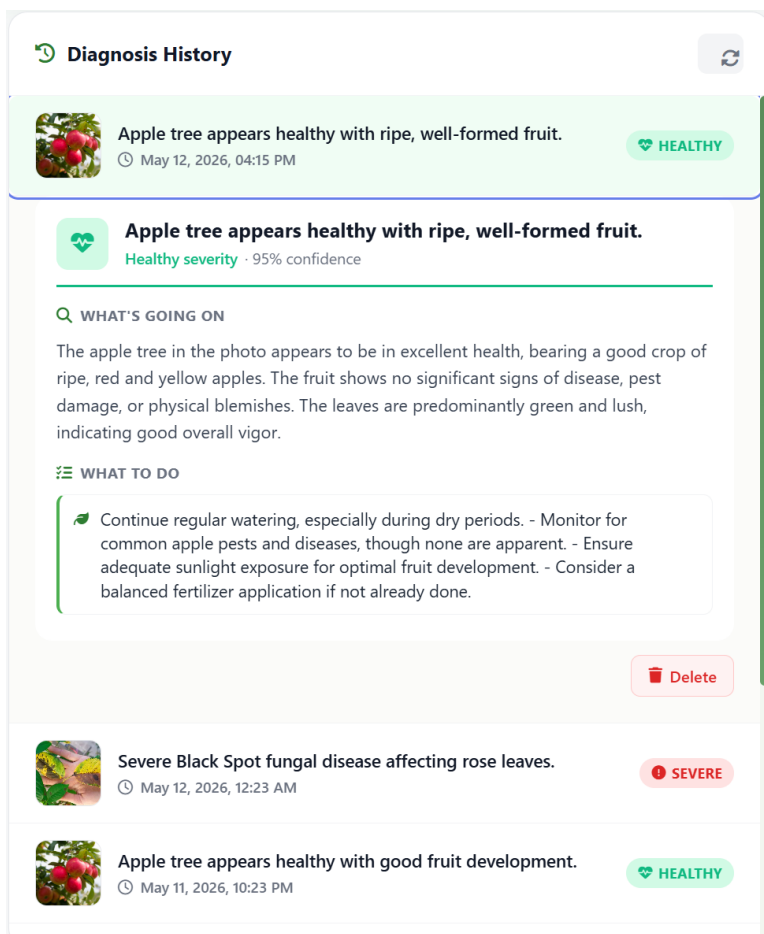
Image	Description	Timestamp	Status
	Apple tree appears healthy with ripe, well-formed fruit.	May 12, 2026, 04:15 PM	HEALTHY
	Severe Black Spot fungal disease affecting rose leaves.	May 12, 2026, 12:23 AM	SEVERE
	Apple tree appears healthy with good fruit development.	May 11, 2026, 10:23 PM	HEALTHY
	Severely dehydrated or dead plant, likely due to extreme lack ...	May 11, 2026, 12:33 AM	SEVERE
	Monstera leaf showing severe discoloration, likely dying d...	May 11, 2026, 12:32 AM	MODERATE
	Severe Black Spot fungal infection on rose leaves causing ...	May 11, 2026, 12:26 AM	MODERATE
	Monstera deliciosa appears healthy with robust new growth.	May 11, 2026, 12:25 AM	HEALTHY

Slika 16: Plant Doctor — sučelje za novu dijagnozu i pregled povijesti

3.8. Diagnosis History — povijest dijagnoza

Svaka obavljena dijagnoza pohranjuje se u povijest dijagnoza s thumbnailom analizirane fotografije, kratkim sažetkom problema, vremenskom oznakom i bedžom razine ozbiljnosti. Korisnik klikom otvara puni zapis koji sadrži dvije sekcije: „What's going on“ (detaljno objašnjenje uočenih simptoma i pretpostavljenog uzroka) i „What to do“ (popis preporučenih radnji za oporavak biljke).

Pored opisa, sustav prikazuje i razinu pouzdanosti dijagnoze (npr. „95% confidence“) te omogućuje brisanje zapisa. Razine ozbiljnosti vizualno se razlikuju bojama: zelena za HEALTHY, žuta za MODERATE i crvena za SEVERE, što olakšava brzi pregled stanja kroz veću kolekciju.

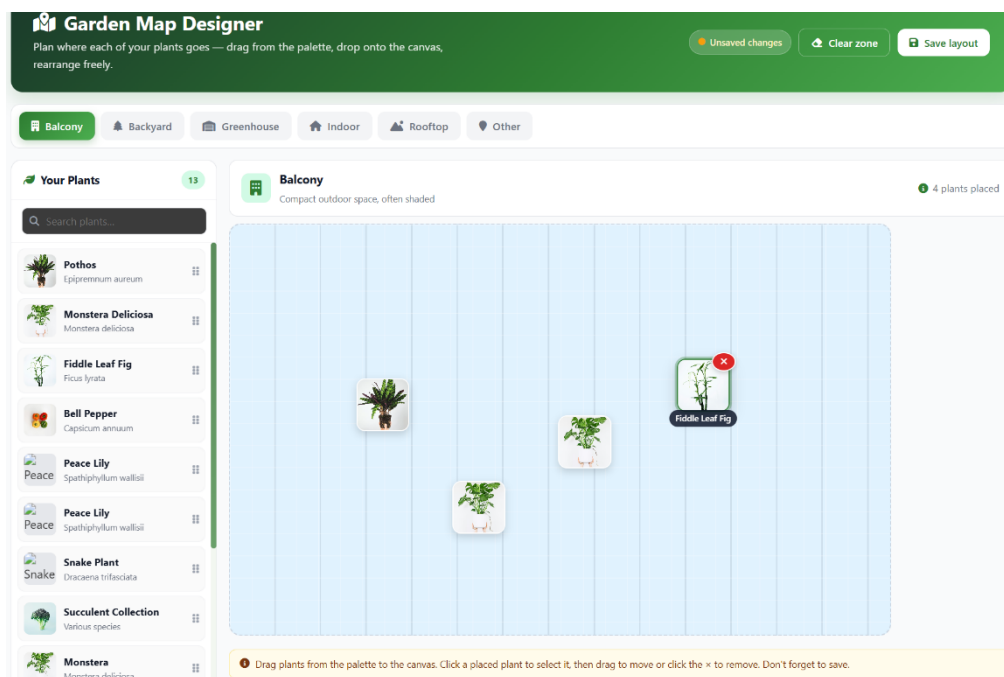


Slika 17: Diagnosis History — detaljan prikaz pojedinačnog zapisa dijagnoze

3.9. Garden Studio — Garden Map Designer

Garden Map Designer je interaktivni planer rasporeda biljaka unutar šest unaprijed definiranih zona: Balcony, Backyard, Greenhouse, Indoor, Rooftop i Other. Korisnik mijenja aktivnu zonu pomoću tabova u zaglavlju, a svaka zona ima vlastiti opis i kapacitet (npr. „Compact outdoor space, often shaded“ za balkon).

S lijeve strane sučelja nalazi se paleta „Your Plants“ s popisom svih biljaka iz kolekcije i tražilicom. Korisnik povlači biljke iz palete na mrežnu canvas površinu (drag-and-drop) i postavlja ih na proizvoljne pozicije. Postavljene biljke moguće je naknadno pomicati, označavati klikom (zelena obruba selekcije) ili ukloniti pritiskom na crveni X. Promjene se prikazuju indikatorom „Unsaved changes“ sve dok korisnik ne klikne „Save layout“, što layout pohranjuje u bazu putem dedicated endpointa.



Slika 18: Garden Map Designer — drag-and-drop raspored biljaka u zoni Balcony

```

const loadZone = async (z) => {
  try {
    setLoadingMap(true);
    setSelectedId(null);
    const res = await apiService.getGardenMap(z);
    if (res.success) {
      const items = (res.data || []).map(p => ({ ...p, _uid: `srv-${p.id}` }));
      setPlacements(items);
      setDirty(false);
    }
  } catch (e) {
    showNotification('Error', 'Could not load the map', 'error');
    setPlacements([]);
  } finally {
    setLoadingMap(false);
  }
};

const handlePaletteDragStart = (plant, e) => {
  dragState.current = {
    mode: 'palette',
    sourcePlant: plant,
    offsetX: TILE_SIZE / 2,
    offsetY: TILE_SIZE / 2,
  };
  e.dataTransfer.effectAllowed = 'copy';
  e.dataTransfer.setData('text/plain', String(plant.id));
};

// — Drag an existing placement around the canvas —
const handlePlacementDragStart = (placement, e) => {

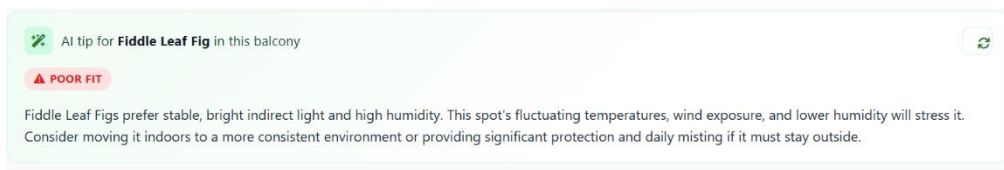
```

Slika 19: GardenMapDesigner.jsx — loadZone i handlePaletteDragStart

3.10. Garden Studio — AI Zone Fit analiza

Za svaku postavljenu biljku unutar zone, sustav generira personalizirani AI savjet — Zone Fit analizu. Savjet procjenjuje odgovara li odabrana zona stvarnim potrebama biljke (svjetlo, vlaga, temperatura, izloženost vjetru) i klasificira rezultat u jednu od tri kategorije: GOOD FIT (zelena), MARGINAL (žuta) i POOR FIT (crvena).

Primjer: Fiddle Leaf Fig postavljen na balkon dobiva oznaku POOR FIT uz objašnjenje da biljka preferira stabilno svjetlo, visoku vlažnost i ne podnosi kolebanja temperature i vjetar. AI prijedlozi obuhvaćaju i konkretne korake (premještanje u zatvoreni prostor, dnevno orošavanje). Ova povratna informacija ključna je za prevenciju gubitka biljaka zbog neprikladne lokacije.

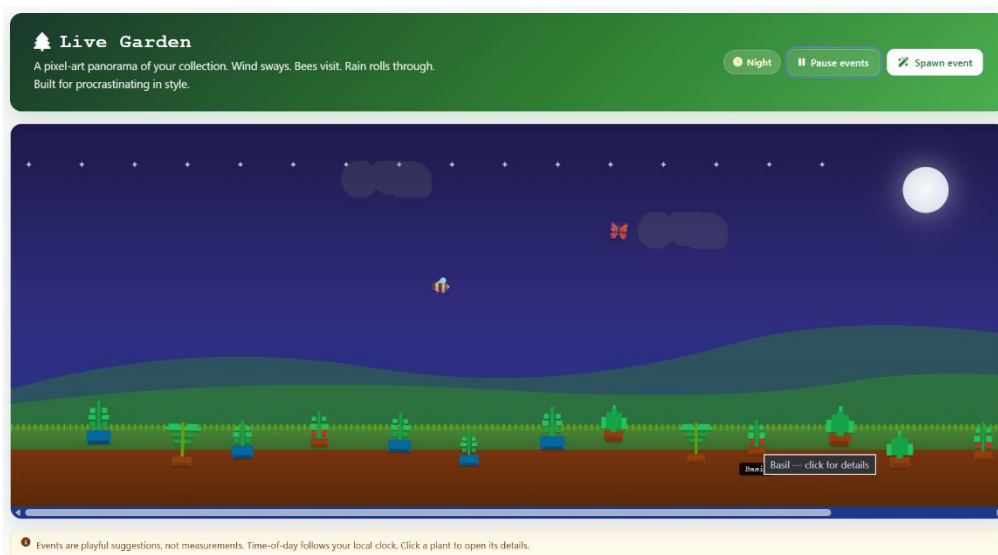


Slika 20: AI Zone Fit — POOR FIT analiza za Fiddle Leaf Fig na balkonu

3.11. Garden Studio — Live Garden panorama

Live Garden je pixel-art panorama korisnikove kolekcije izgrađena kao SVG/canvas vizualizacija. Sve biljke iz kolekcije prikazane su poredane u redu na površini tla s ogradom u prvom planu i brežuljkastim horizontom u pozadini. Vrijeme dana u panorami prati lokalni sat korisnika — preko dana prikazuje se vedro nebo i sunce, a noću zvjezdano nebo i mjesec.

Sustav događaja (events) simulira život vrta: pčele lete od biljke do biljke, leptiri prolaze nebom, vjetar lagano njiše biljke, a povremeno padne i kiša. Korisniku su dostupne kontrole „Night“ za ručno prebacivanje između dana i noći, „Pause events“ za zaustavljanje animacija i „Spawn event“ za ručno aktiviranje slučajnog događaja. Klikom na pojedinu biljku otvara se njena tooltip kartica s nazivom, a dvostruki klik vodi na stranicu detalja.



Slika 21: Live Garden — pixel-art panorama s noćnim ciklusom i animacijama

```

const pickEvent = (plants) => {
  const viable = EVENT_TEMPLATES.filter(t => !t.plantFilter || plants.some(t.plantFilter));
  if (viable.length === 0) return null;
  const totalW = viable.reduce((s, t) => s + t.weight, 0);
  let roll = Math.random() * totalW;
  let template = viable[0];
  for (const t of viable) { roll -= t.weight; if (roll <= 0) { template = t; break; } }
  const candidatePlants = template.plantFilter ? plants.filter(template.plantFilter) : plants;
  const plant = candidatePlants[Math.floor(Math.random() * candidatePlants.length)] || null;

  const body = template.body.replace('$PLANT', plant?.name || 'one of your plants');
  return {
    uid: `${template.id}-${Date.now()}-${Math.random().toString(36).slice(2,7)}`,
    icon: template.icon,
    title: template.title,
    body,
    plantId: plant?.id || null,
  };
};

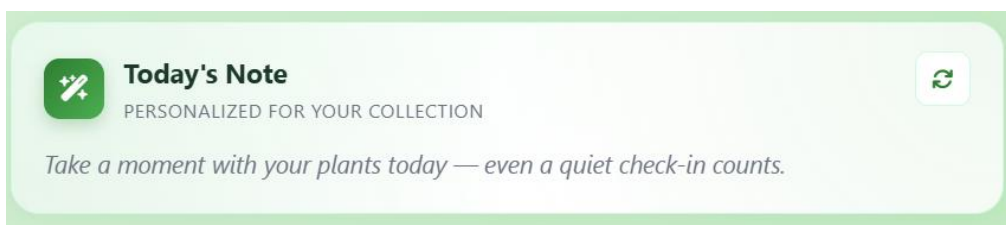
```

Slika 22: LiveGarden.jsx — pickEvent funkcija s weighted random odabirom

3.12. Today's Note — personalizirana dnevna poruka

Komponenta Today's Note pojavljuje se na Dashboardu i prikazuje kratku personaliziranu poruku za korisnika („Personalized for your collection“). Tekst se generira AI modelom na temelju trenutnog stanja kolekcije: broja biljaka, statusa zadataka njege, vremenskih uvjeta i sezone. Cilj je potaknuti korisnika na svakodnevnu interakciju s aplikacijom čak i kada nema akutnih zadataka — npr. „Take a moment with your plants today — even a quiet check-in counts.“

Korisnik može osvježiti notu klikom na ikonu refresha u gornjem desnom kutu kartice, čime se zahtijeva novi prijedlog. Notu generira backend endpoint koji koristi kontekst korisničkih biljaka i statistika za izradu prompta poslanog AI servisu.

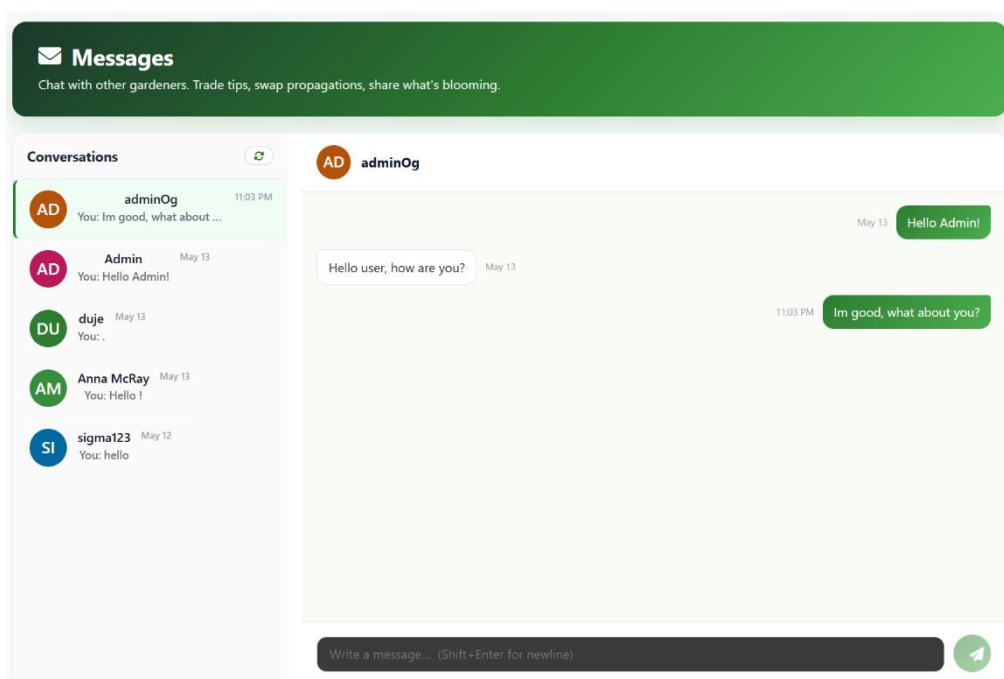


Slika 23: Today's Note — personalizirana poruka na Dashboardu

3.13. Direct Messages — privatne poruke među korisnicima

Modul Messages omogućuje privatnu komunikaciju među korisnicima aplikacije. Sučelje je podijeljeno na dva panela: lijevo se nalazi popis razgovora (Conversations) s avatarima sugovornika, posljednjom porukom i vremenskom oznakom, a desno aktivni razgovor s povijesti poruka. Vlastite poruke prikazuju se desno u zelenim mjehurićima, a primljene lijevo u bijelim.

Korisnik pokreće razgovor s drugim korisnikom kroz Gardeners modul (gumb „Send message“ na profilu) ili odabirom iz postojeće liste razgovora. Polje za unos podržava Shift+Enter za novi red, a poruke se šalju kroz dedicated backend endpoint messages.php koji pohranjuje zapise u tablicu messages s poljima id, sender_id, recipient_id, content, created_at i is_read. Refresh dugme u zaglavlju liste razgovora osvježava status pristiglih poruka.



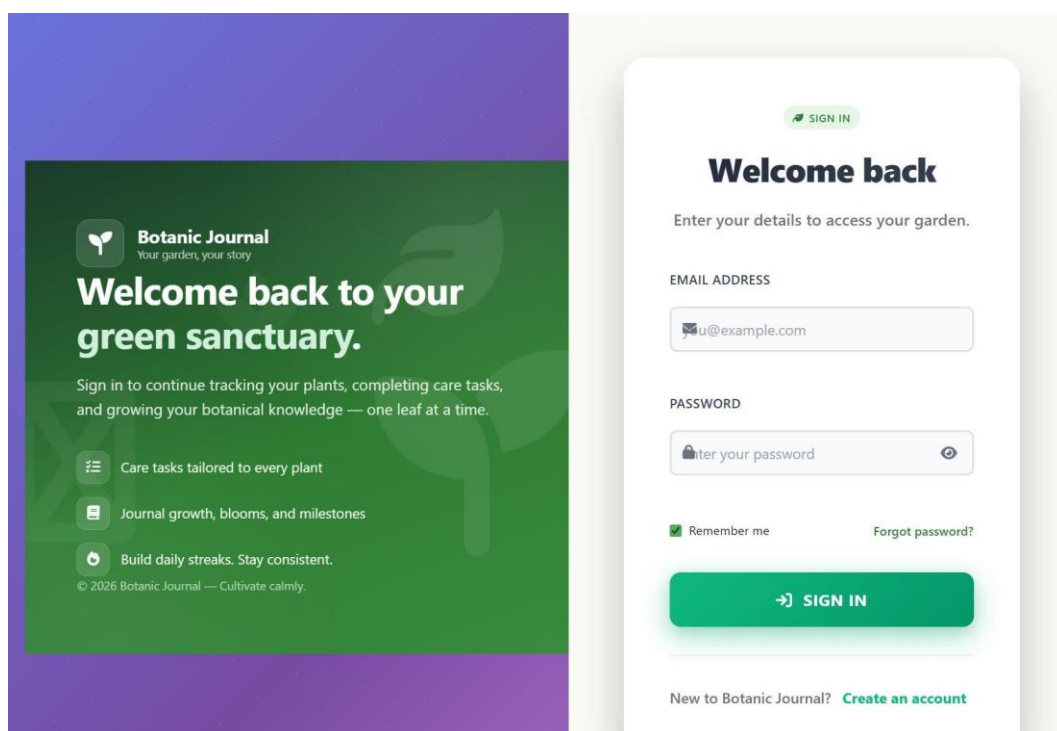
Slika 24: Direct Messages — sučelje za privatne razgovore među korisnicima

4. Backend implementacija

Backend je implementiran kao skup PHP skripti koje zajedno čine REST API. Svaka skripta prima HTTP zahtjeve, obavlja operacije nad bazom podataka putem PDO sučelja i vraća JSON odgovore.

4.1. Autentikacija (auth.php)

Autentikacijski endpoint podržava dvije akcije: register i login. Registracija validira email format, provjerava duplikate u bazi te pohranjuje lozinku kao bcrypt hash putem password_hash() funkcije. Prijava koristi password_verify() za usporedbu unesene lozinke s hashom iz baze.



Slika 25: Login stranica — redizajnirani two-panel prikaz

GET STARTED

Create your account

Free forever. No credit card required.

FULL NAME

John Doe

EMAIL ADDRESS

johnndoe@gmail.com

PASSWORD

.....

Password strength: Strong

CONFIRM PASSWORD

.....

Slika 26: Registracijska forma s provjerom jakosti lozinke

Uspješna prijava/registracija vraća korisničke podatke bez lozinke. Polje 'name' iz baze mapira se na 'username' za kompatibilnost s frontendom. Korisnici imaju role: 'user' (zadano) ili 'admin'.

4.2. Upravljanje biljkama (plants.php)

Plants.php je najkompleksniji endpoint koji rukuje svim CRUD operacijama nad biljkama. GET zahtjev bez parametara vraća sve biljke korisnika; s parametrom ?encyclopedia vraća enciklopedijske biljke. POST kreira novu biljku - ako se šalje encyclopedia_id, kopira podatke iz enciklopedijske biljke.

Posebna funkcija `fixUserPlants()` automatski ispravlja biljke koje su pogrešno označene kao enciklopedijske (`is_encyclopedia = 1`) a zapravo su u vlasništvu korisnika. Ova funkcija se poziva pri svakom GET zahtjevu kako bi se spriječili problemi s prikazom.

```
backend > api > plants.php > handleGetRequest
118 function fixUserPlants($db, $user_id) {
119     error_log('Fixed $fixed_count plants for user $user_id (set is_encyclopedia = 0)');
120 }
121
122
123     return $fixed_count;
124 }
125
126 function handleGetRequest($db, $user_id)
127 {
128     // Auto-fix user plants on every GET request
129     $fixed_count = fixUserPlants($db, $user_id);
130
131     if (isset($_GET['id'])) {
132         // Get single plant
133         $stmt = $db->prepare("SELECT * FROM plants WHERE id = :id AND user_id = :user_id");
134         $stmt->bindParam(':id', $_GET['id']);
135         $stmt->bindParam(':user_id', $user_id);
136         $stmt->execute();
137         $plant = $stmt->fetch(PDO::FETCH_ASSOC);
138
139         if (!$plant) {
140             sendError('Plant not found', 404);
141         }
142
143         sendSuccess($plant, 'Plant retrieved successfully');
144     } else if (isset($_GET['encyclopedia'])) {
145         // Get encyclopedia plants with added status
146         $stmt = $db->prepare("
147             SELECT p.*,
148                    (SELECT COUNT(*) FROM plants up
149                     WHERE up.encyclopedia_id = p.id AND up.user_id = :user_id
150                     AND up.is_encyclopedia = 0) as is_added
151             FROM plants p
152         ");
153         $stmt->bindParam(':user_id', $user_id);
154         $stmt->execute();
155         $plants = $stmt->fetchAll(PDO::FETCH_ASSOC);
156         sendSuccess($plants, 'Plants retrieved successfully');
157     }
158 }
```

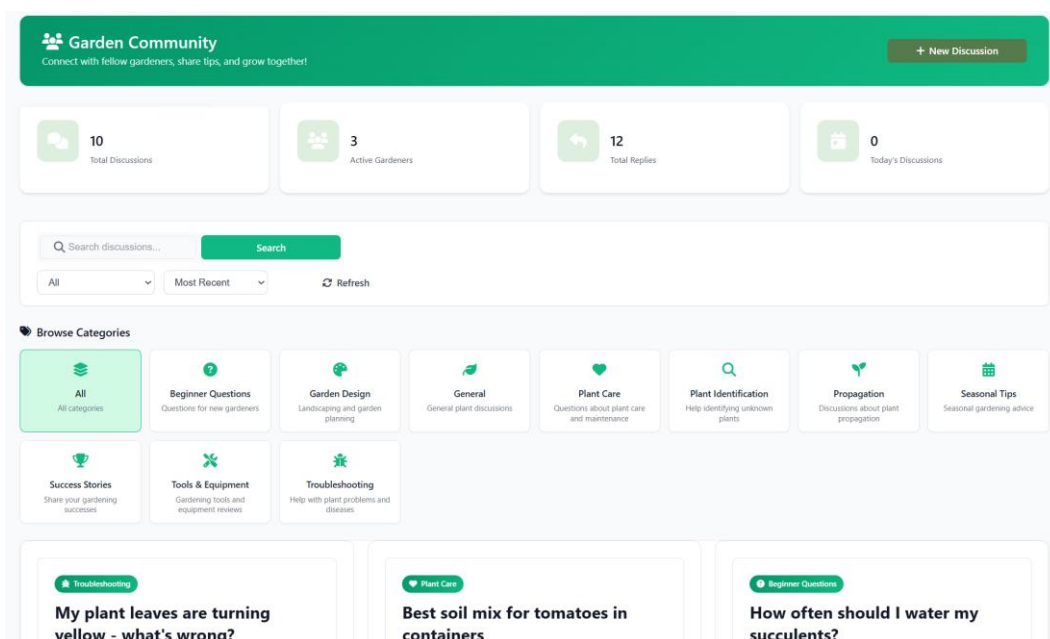
Slika 27: `plants.php` - PDO prepared statements

4.3. Community forum (community/)

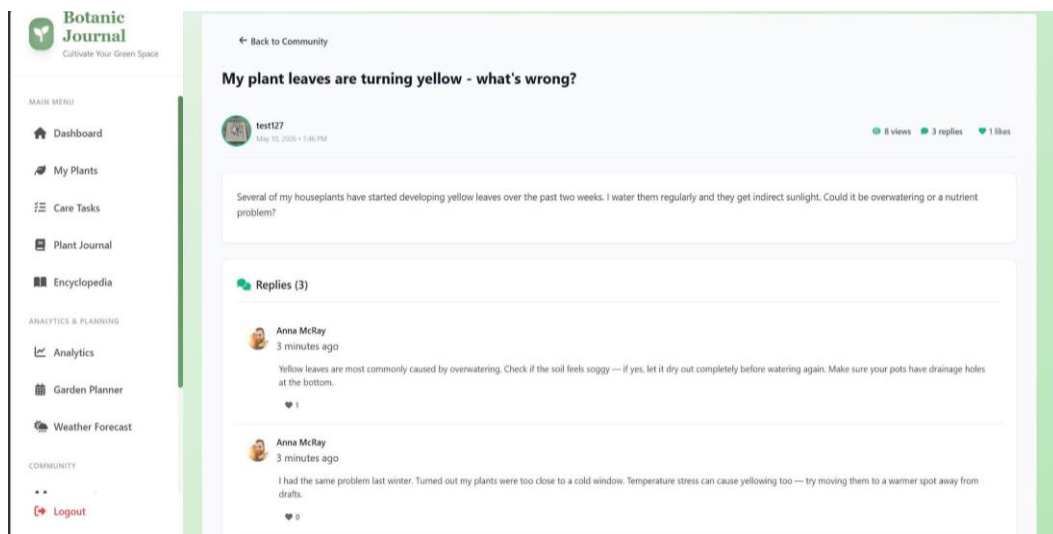
Community backend sadrži pet endpointa:

- `discussions.php` - CRUD za rasprave s filteriranjem po kategoriji, pretraživanjem i paginacijom
- `replies.php` - kreiranje i dohvaćanje odgovora s informacijama o autoru
- `likes.php` - dodavanje/uklanjanje lajkova na raspravama i odgovorima (POST/DELETE)
- `categories.php` - dohvaćanje popisa kategorija foruma
- `stats.php` - statistike foruma (ukupno rasprava, odgovora, aktivnih korisnika, today)

Model `CommunityDiscussion.php` implementira metode `readAll()` s opcionalnim filtriranjem, `readOneWithUser()` koja dohvaća i status lajka prijavljenog korisnika, te `like()/unlike()` metode koje koriste zasebnu tablicu `discussion_likes`.

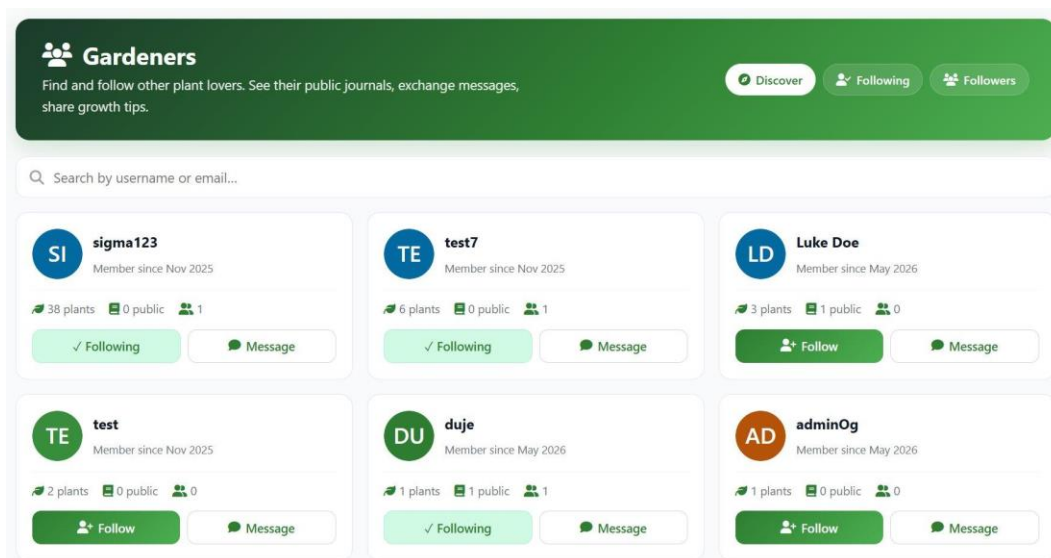


Slika 28: Community Forum - statistike i kategorije

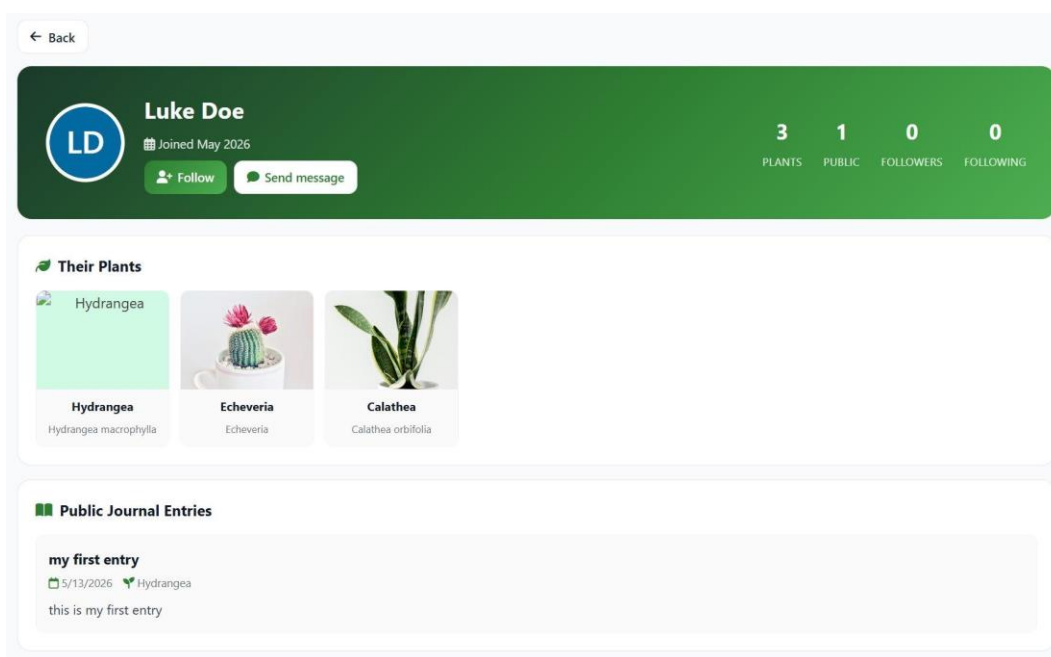


Slika 29: Discussion thread s odgovorima

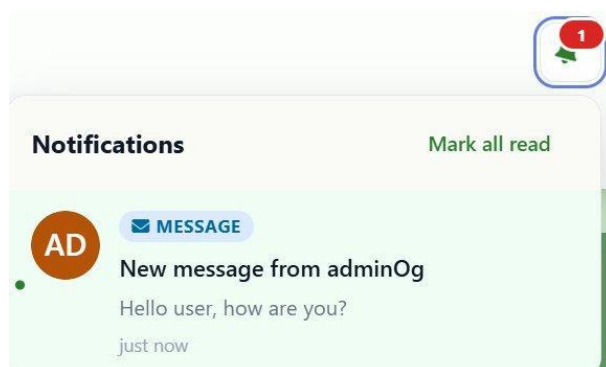
Unutar Community sekcije dostupan je i modul Gardeners koji korisnicima omogućuje pronalazak i praćenje drugih ljubitelja biljaka. Prikazuju se kartice korisnika s brojem biljaka, javnih objava i pratitelja. Korisnik može pratiti druge (Follow/Unfollow), slati direktne poruke i pregledavati javni profil s kolekcijom biljaka i javnim journal unosima.



Slika 30: Gardeners — pretraživanje i praćenje ostalih korisnika



Slika 31: Javni korisnički profil s biljkama i journal unosima



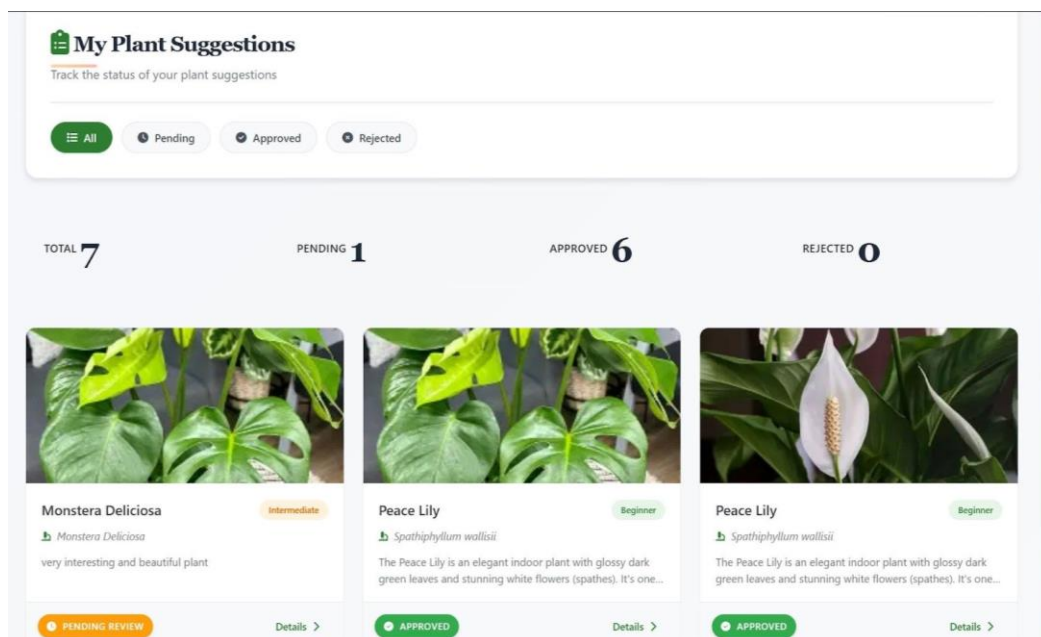
Slika 32: Notifikacijski sustav — realtime obavijesti

4.4. Plant Requests sustav

Plant Requests je sustav koji korisnicima omogućuje predlaganje novih biljaka za enciklopediju. Korisnik ispunjava multi-step formu s osnovnim informacijama, uputama za njegu i fotografijama. Administrator može prihvatiti ili odbiti prijedlog.

Slika 33: Suggest New Plant — korak 1 Basic Info

Pri prihvaćanju prijedloga, plant-requests.php automatski kreira novu enciklopedijsku biljku u tablici plants mapiranjem podataka iz zahtjeva (difficulty_level mapira u Easy/Moderate/Advanced, growth_rate u Slow/Moderate/Fast). Promjene se izvode u transakciji (BEGIN/COMMIT) za konzistentnost podataka.

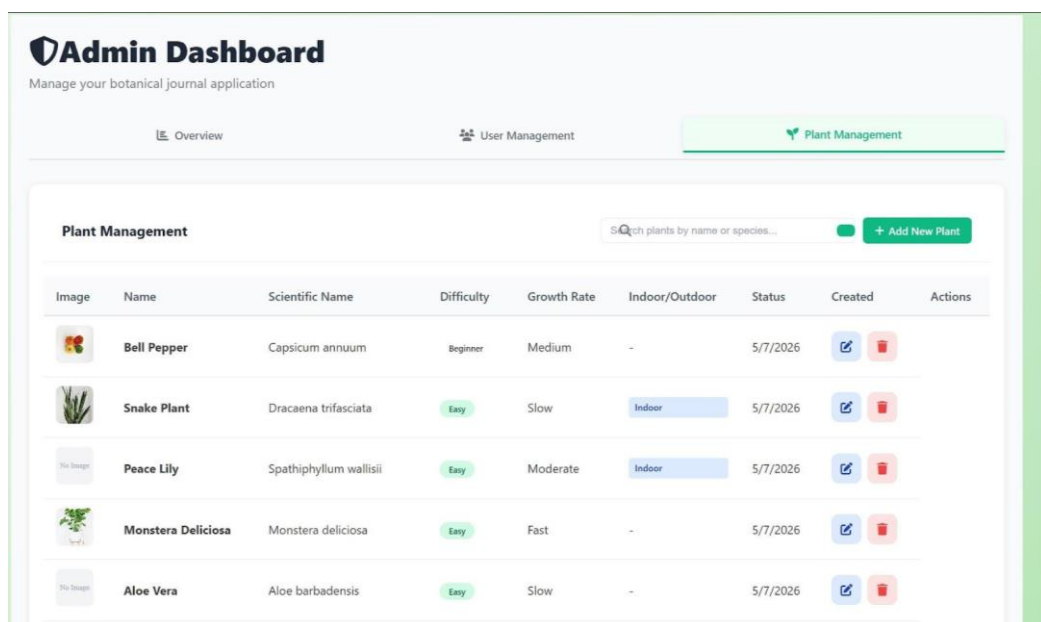


Slika 34: My Plant Suggestions — pregled statusa prijedloga

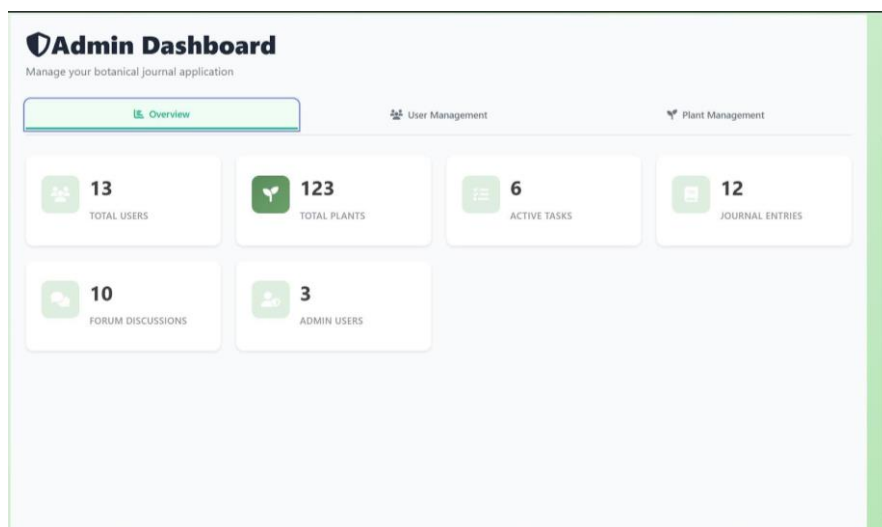
4.5. Admin panel

Admin backend (admin/) sadrži tri krajnje točke(endpointa):

- plants.php - upravljanje enciklopedijskim biljkama: pregled, kreiranje (s uploadom slika), uređivanje, brisanje



Slika 35: Admin Dashboard - Plant Management



Slika 36: Admin Dashboard - Overview statistike

- stats.php - ukupni pregled: broj korisnika, biljaka, zadataka, dnevnika, rasprava

Admin Dashboard
Manage your botanical journal application

Overview | **User Management** | Plant Management

User Management | Export Users

USER	EMAIL	ROLE	LEVEL	STATUS	MEMBER SINCE	LAST LOGIN	ACTIONS
JP John Pork	johnpork@gmail.com		1		5/10/2026	5/10/2026	
AM Anna McRay	annathegardener@gmail.com		1		5/10/2026	5/10/2026	
AD adminOg	admin127@gmail.com		1		5/4/2026	5/10/2026	
AU Admin User	admin1@botanicjournal.com		1		4/5/2026	4/10/2026	
IU iuser	user@example.com		1		3/31/2026	3/31/2026	

Slika 37: Admin Dashboard - User Management

- users.php - upravljanje korisnicima: pregled, promjena role, aktivacija/deaktivacija, brisanje

Svi admin endpointi provjeravaju je li zahtjev od korisnika s role = 'admin' u bazi. Administrator ne može mijenjati/brisati vlastiti račun (safety check).

4.6. Upload datoteka

Sustav podržava upload slika na tri mjesta: avatar korisnika (update-avatar.php), slike biljaka pri admin kreiranju i slike uz plant requests.

Svaki upload validira tip datoteke (jpg, jpeg, png, webp, gif), maksimalnu veličinu (5 MB), te sprema datoteku s jedinstvenim imenom (time() + uniqid()) u odgovarajuću mapu unutar /backend/uploads/.

4.7. Plant Doctor API (plant-doctor.php)

plant-doctor.php je AI dijagnostički endpoint koji prima multipart upload sa slikom biljke. POST zahtjev s poljima image (datoteka), opcionalnim notes (opis simptoma) i plant_id (vezivanje uz biljku iz kolekcije) prosljeđuje sliku Gemini API-ju, parsira strukturirani odgovor i pohranjuje rezultat u tablicu plant_diagnoses.

Strukturirani odgovor sadrži polja ai_summary (kratki sažetak), ai_diagnosis (detaljno objašnjenje), ai_recommendations (popis preporuka), ai_severity (ENUM: healthy, mild, moderate, severe, unknown) i ai_confidence (0–100). GET vraća popis svih dijagnoza za korisnika ili pojedinačnu po id-u; DELETE briše zapis i pripadnu datoteku slike iz /backend/uploads/diagnoses/.

```

function handleDiagnose($db, $user_id) {
    $plant_context = '';
    if ($plant_id) {
        $stmt = $db->prepare("SELECT name, species, type, light_requirements, watering_schedule
                              FROM plants WHERE id = :id AND user_id = :uid");
        $stmt->execute([':id' => $plant_id, ':uid' => $user_id]);
        $p = $stmt->fetch(PDO::FETCH_ASSOC);
        if ($p) {
            $plant_context = "\nPlant context (from user's collection): "
                . "Name: {$p['name']}; Species: {$p['species']}; Type: {$p['type']}; "
                . "Light: {$p['light_requirements']}; Watering: {$p['watering_schedule']}. ";
        }
    }

    $analysis = callGeminiVision($disk_path, $ext, $notes, $plant_context);
    $stmt = $db->prepare("INSERT INTO plant_diagnoses
                        (user_id, plant_id, image_path, user_notes, ai_summary, ai_diagnosis,
                         ai_recommendations, ai_severity, ai_confidence, raw_response, created_at)
                        VALUES (:uid, :pid, :img, :notes, :sum, :diag, :rec, :sev, :conf, :raw, NOW())");
    $stmt->execute([
        ':uid' => $user_id,
        ':pid' => $plant_id,
        ':img' => $public_url,
        ':notes' => $notes,
        ':sum' => $analysis['summary'],
        ':diag' => $analysis['diagnosis'],
        ':rec' => $analysis['recommendations'],
        ':sev' => $analysis['severity'],
        ':conf' => $analysis['confidence'],
        ':raw' => $analysis['raw'],
    ]);
    $newId = $db->lastInsertId();
    $row = $db->prepare("SELECT * FROM plant_diagnoses WHERE id = :id");
    $row->execute([':id' => $newId]);
    respond(true, 'Diagnosis complete', $row->fetch(PDO::FETCH_ASSOC), 201);
}

```

Slika 38: *plant-doctor.php* — *handleDiagnose* s Gemini Vision API integracijom

4.8. Plant Assistant API (*plant-chat.php*)

plant-chat.php implementira backend za AI chat asistenta (Plant Assistant). POST zahtjev s poljem *message* šalje upit Gemini modelu zajedno s kontekstom korisničke kolekcije (popis biljaka, njihovi statusi, posljednji zadaci njege). Par korisničke poruke i odgovora pohranjuje se u tablicu *plant_chat_messages* s ENUM poljem *role* (*user/assistant*), a razgovori se grupiraju u *plant_chat_conversations*. Time se omogućuje povijest razgovora kroz više sesija.

4.9. Garden Map placements (garden-map.php)

garden-map.php upravlja rasporedom biljaka u zonama vrta. GET ?zone=balcony vraća sve placement-e zone (ako se zone izostavi, vraća sve zone korisnika). POST s body { zone, placements: [...] } atomarno zamjenjuje cijeli raspored zone (transakcija DELETE postojeće → INSERT nove). DELETE briše sve placement-e zone. Tablica garden_map_placements pohranjuje x_pos, y_pos, size, rotation i z_index kao numeričke vrijednosti, što omogućuje precizno reprodukciranje drag-and-drop layouta na klijentu.

```
function handleSave($db, $user_id, $allowedZones) {
    $body = json_decode(file_get_contents('php://input'), true);
    if (!is_array($body)) respond(false, 'Invalid JSON body', null, 400);

    $zone = isset($body['zone']) ? strtolower(trim($body['zone'])) : '';
    if (!in_array($zone, $allowedZones)) respond(false, 'Invalid or missing zone', null, 400);

    $placements = isset($body['placements']) && is_array($body['placements']) ? $body['placements'] : [];

    $db->beginTransaction();
    try {
        // Wipe existing for this zone
        $del = $db->prepare("DELETE FROM garden_map_placements WHERE user_id = :uid AND zone = :z");
        $del->execute([':uid' => $user_id, ':z' => $zone]);

        $ins = $db->prepare("INSERT INTO garden_map_placements
            (user_id, zone, plant_id, x_pos, y_pos, size, rotation, z_index, note)
            VALUES (:uid, :z, :pid, :x, :y, :sz, :rot, :zi, :note)");

        foreach ($placements as $idx => $p) {
            if (!isset($p['plant_id']) || !isset($p['x_pos']) || !isset($p['y_pos'])) continue;
            $ins->execute([
                ':uid' => $user_id,
                ':z' => $zone,
                ':pid' => intval($p['plant_id']),
                ':x' => floatval($p['x_pos']),
                ':y' => floatval($p['y_pos']),
                ':sz' => isset($p['size']) ? max(32, min(200, intval($p['size']))) : 72,
                ':rot' => isset($p['rotation']) ? intval($p['rotation']) : 0,
                ':zi' => isset($p['z_index']) ? intval($p['z_index']) : $idx,
                ':note' => isset($p['note']) ? mb_substr($p['note'], 0, 200) : null,
            ]);
        }
    }
}
```

Slika 39: garden-map.php — handleSave atomarna transakcija (DELETE + INSERT)

4.10. AI Zone Fit (garden-map-tip.php)

garden-map-tip.php generira AI procjenu kompatibilnosti biljke i odabrane zone. Endpoint prima plant_id i zone, dohvaća karakteristike biljke (svjetlosni zahtjevi, vlažnost, temperaturni raspon) i šalje upit AI modelu koji vraća verdict (ENUM: great, okay, poor, unknown) i tekstualni tip s konkretnim preporukama. Rezultati se cache-iraju u tablicu garden_map_tips po paru (plant_id, zone) — ako tip već postoji, vraća se postojeći umjesto novog poziva AI-ja, čime se smanjuje broj API poziva i ubrzava prikaz.

```
try {
    $systemText =
        "You are a horticulturalist giving a quick microclimate fit assessment for ONE plant " .
        "in ONE specific spot of a user's garden. Respond with a JSON object with these exact keys:\n" .
        "\"verdict\": one of 'great' | 'okay' | 'poor' | 'unknown' - how well this plant fits this microclimate.\n" .
        "\"tip\": a single short paragraph (2-3 sentences, max ~60 words) of practical advice " .
        "specific to this plant in this location. Be concrete. Examples:\n" .
        "- If a great fit: highlight why and what to monitor.\n" .
        "- If poor: warn what will go wrong AND suggest a small mitigation (e.g. 'mist daily', 'move to shade in summer').\n" .
        "Plain text, no markdown, no emoji. Never invent species details.";

    $plantSummary =
        "Plant: " . $plant['name'] .
        (!empty($plant['species']) ? " (" . $plant['species'] . ")": "") .
        "\nType: " . ($plant['type'] ?? 'unknown') .
        "\nLight needs: " . ($plant['light_requirements'] ?? 'unknown') .
        "\nWater needs: " . ($plant['watering_schedule'] ?? 'unknown') .
        "\nTemperature range: " . ($plant['temperature_range'] ?? 'unknown') .
        "\nHumidity needs: " . ($plant['humidity_requirements'] ?? 'unknown');

    $userText = "Microclimate of the chosen spot:\n$zoneDesc\n\n$plantSummary";

    $responseSchema = [
        'type' => 'object',
        'properties' => [
            'verdict' => ['type' => 'string', 'enum' => ['great', 'okay', 'poor', 'unknown']],
            'tip' => ['type' => 'string'],
        ],
        'required' => ['verdict', 'tip'],
    ];
}
```

Slika 40: garden-map-tip.php — generiranje AI Zone Fit savjeta s cache-om

4.11. Today's Note (daily-care-note.php)

daily-care-note.php upravlja personaliziranom dnevnom porukom za korisnika. GET zahtjev vraća današnju notu; ako za današnji datum ne postoji zapis, endpoint generira novu putem AI poziva s kontekstom korisničke kolekcije i trenutnih vremenskih uvjeta, pohranjuje je u tablicu daily_care_notes i vraća korisniku. UNIQUE constraint na paru (user_id, for_date) jamči samo jednu notu po danu po korisniku, čime se sprečavaju duplikati pri istovremenim zahtjevima.

4.12. Direct Messages (direct-messages.php)

direct-messages.php implementira sustav privatnih poruka 1:1. GET bez parametara vraća popis svih razgovora korisnika (s najnovijom porukom, brojem nepročitanih i podacima o sugovorniku); GET s parametrom with=Y vraća punu povijest razgovora s korisnikom Y (kronološki). POST s body { to, content } sprema novu poruku u tablicu direct_messages i automatski kreira notifikaciju za primatelja. PATCH označava sve poruke od korisnika from kao pročitane postavljanjem read_at vrijednosti.

```
function handleGet($db, $user_id) {  
    if (isset($_GET['with'])) {  
        return getConversation($db, $user_id, intval($_GET['with']));  
    }  
    return listConversations($db, $user_id);  
}
```

Slika 41: direct-messages.php — handleGet entry point

4.13. Social — follow/unfollow (social.php)

social.php upravlja sustavom praćenja korisnika. Endpoint koristi parametar action koji određuje operaciju: discover (popis svih korisnika za pretragu, isključujući trenutnog), following (popis korisnika koje korisnik prati), followers (popis pratitelja), profile&target=Y (javni profil korisnika Y s njegovim biljkama, javnim journal unosima i statistikama). POST kreira novi follow zapis u tablici user_follows, DELETE ga briše. UNIQUE constraint sprečava duplo praćenje istog korisnika.

```
function getPublicProfile($db, $viewer_id) {
    $target = isset($_GET['target']) ? intval($_GET['target']) : null;
    if (!$target) respond(false, 'target user id required', null, 400);

    $stmt = $db->prepare("SELECT id, name AS username, email, avatar, created_at, role
                        FROM users WHERE id = :id");
    $stmt->execute([':id' => $target]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);
    if (!$user) respond(false, 'User not found', null, 404);

    $plants_count = (int)$db->query("SELECT COUNT(*) FROM plants WHERE user_id = $target")->fetchColumn();
    $journals_count = (int)$db->query("SELECT COUNT(*) FROM journals WHERE user_id = $target AND is_public = 1")->fetchColumn();
    $followers_count = (int)$db->query("SELECT COUNT(*) FROM user_follows WHERE followed_id = $target")->fetchColumn();
    $following_count = (int)$db->query("SELECT COUNT(*) FROM user_follows WHERE follower_id = $target")->fetchColumn();

    $check = $db->prepare("SELECT 1 FROM user_follows WHERE follower_id = :v AND followed_id = :t");
    $check->execute([':v' => $viewer_id, ':t' => $target]);
    $is_following = (bool)$check->fetchColumn();

    $stmt = $db->prepare("SELECT id, name, species, type, image_url, status
                        FROM plants WHERE user_id = :uid
                        ORDER BY created_at DESC LIMIT 12");
    $stmt->execute([':uid' => $target]);
    $plants = $stmt->fetchAll(PDO::FETCH_ASSOC);
}
```

Slika 42: social.php — discoverUsers i pomoćne funkcije za listanje

```
function handleGet($db, $user_id) {
    $action = isset($_GET['action']) ? $_GET['action'] : 'discover';

    switch ($action) {
        case 'discover': return discoverUsers($db, $user_id);
        case 'following': return listFollowing($db, $user_id);
        case 'followers': return listFollowers($db, $user_id);
        case 'profile': return getPublicProfile($db, $user_id);
        default: return respond(false, 'Unknown action', null, 400);
    }
}
```

Slika 43: social.php — handleGet switch po action parametru

5. Sigurnosni aspekti

Sigurnost web aplikacija od ključne je važnosti, posebno kada se radi s korisničkim podacima. Ovaj projekt implementira nekoliko slojeva zaštite, premda postoje i aspekti koji bi trebali biti poboljšani u produkcijskoj verziji.

5.1. Zaštita lozinki

Lozinke se nikada ne pohranjuju u čistom tekstu. Auth.php koristi PHP-ovu `password_hash()` funkciju s `PASSWORD_DEFAULT` algoritmom (bcrypt) koji automatski generira salt i stvara siguran hash. Verifikacija se obavlja putem `password_verify()` funkcije. Ovaj pristup sprečava kompromitiranje lozinki čak i u slučaju curenja baze podataka.

```
case 'register':
    if (!isset($data['name']) || !isset($data['email']) || !isset($data['password'])) {
        throw new Exception('All fields are required');
    }
    $name = trim($data['name']);
    $email = trim($data['email']);
    $password = $data['password'];
    $role = 'user';
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new Exception('Invalid email format');
    }

    $check_stmt = $db->prepare("SELECT id FROM users WHERE email = :email");
    $check_stmt->bindParam(':email', $email);
    $check_stmt->execute();

    if ($check_stmt->fetch()) {
        throw new Exception('Email already exists');
    }
    $hashed_password = password_hash($password, PASSWORD_DEFAULT);
```

Slika 44: auth.php — register slučaj s `password_hash()` i validacijom

5.2. SQL Injection zaštita

Sve interakcije s bazom podataka koriste PDO (PHP Data Objects) s pripremljenim izjavama (prepared statements) i vezanjem parametara (bindParam/bindValue). Ovo sprečava SQL injection napade jer korisničke unose nikada ne konkatenujemo direktno u SQL upit.

```
$check_stmt = $db->prepare("SELECT id FROM users WHERE email = :email");
$check_stmt->bindParam(':email', $email);
$check_stmt->execute();

if ($check_stmt->fetch()) {
    throw new Exception('Email already exists');
}
```

Slika 45: auth.php — primjer sigurnog PDO upita s bindParam (SQL Injection zaštita)

5.3. CORS politika

Cross-Origin Resource Sharing (CORS) zaglavlja su konfigurirana tako da dopuštaju zahtjeve samo s <http://localhost:5173> (Vite razvojni server). U produkciji bi ovo trebalo biti postavljeno na stvarnu domenu aplikacije. CORS konfiguracija je centralizirana u `/backend/config/cors.php`.

```
header('Access-Control-Allow-Origin: ' . ($_ENV['FRONTEND_URL'] ?? 'http://localhost:5173'));
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] == 'OPTIONS') {
    http_response_code(200);
    exit();
}
```

Slika 46: cors.php — konfiguracija dopuštenih origina (CORS politika)

5.4. Autorizacija

Svaki zahtjev koji modificira podatke provjerava je li `user_id` koji se šalje stvarno vlasnik resursa. Npr. `plants.php` pri ažuriranju biljke prvo provjerava (`SELECT id FROM plants WHERE id = :id AND user_id = :user_id`), a admin endpointi provjeravaju `role = 'admin'` u bazi prije svake operacije.

```
function checkAdminAuth($db)
{
    $user_id = $_GET['user_id'] ?? null;

    if (!$user_id) {
        http_response_code(401);
        echo json_encode(['success' => false, 'message' => 'Unauthorized']);
        exit();
    }

    $stmt = $db->prepare("SELECT id, role FROM users WHERE id = :id AND is_active = 1");
    $stmt->bindParam(':id', $user_id);
    $stmt->execute();
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$user || $user['role'] !== 'admin') {
        http_response_code(401);
        echo json_encode(['success' => false, 'message' => 'Unauthorized: Admin access required']);
        exit();
    }
}
```

Slika 47: `admin/plants.php` — provjera administratorskih ovlasti prije izvršavanja operacija nad biljkama

ZAKLJUČAK

U ovom završnom stručnom radu razvijen je Botanic Journal - cjelovita full-stack web aplikacija za praćenje i njegu biljaka. Projekt obuhvaća frontend implementiran u React 19.2, backend REST API u PHP-u i MySQL relacijsku bazu podataka.

Kroz razvoj ovog projekta primijenjene su suvremene web tehnologije i koncepti: komponentni pristup razvoju korisničkog sučelja, RESTful API dizajn, relacijski model podataka s referencijalnim integritetom, responzivni dizajn prilagođen svim uređajima, te sigurno rukovanje korisničkim podacima.

Aplikacija uspješno ispunjava sve postavljene ciljeve: korisnici se mogu registrirati i prijaviti, graditi osobnu kolekciju biljaka, dodavati zadatke njege s praćenjem napretka, pisati dnevničke unose, pregledavati detaljnu enciklopediju biljaka, sudjelovati u community forumu, pratiti analitiku kroz grafikone te mogu komunicirati s drugim korisnicima. Administratori imaju poseban panel za upravljanje korisnicima, biljkama i prijedlozima.

LITERATURA

- [1] React Team. React Documentation. Meta Platforms, 2024. <https://react.dev>
- [2] PHP Group. PHP Manual. PHP Documentation Group, 2024. <https://www.php.net/manual/>
- [3] MySQL AB. MySQL 8.0 Reference Manual. Oracle Corporation, 2024. <https://dev.mysql.com/doc/>
- [4] Evan You. Vite Documentation. Vite Team, 2024. <https://vitejs.dev/guide/>
- [5] Chart.js Contributors. Chart.js Documentation. Chart.js Team, 2024. <https://www.chartjs.org/docs/>
- [6] FullCalendar. FullCalendar Documentation. 2024. <https://fullcalendar.io/docs>
- [7] MDN Web Docs. Fetch API. Mozilla Developer Network, 2024. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [8] WeatherAPI. WeatherAPI Documentation. 2024. <https://www.weatherapi.com/docs/>
- [9] npm. docx - Generate .docx files with JavaScript. 2024. <https://www.npmjs.com/package/docx>
- [10] OWASP. OWASP Top Ten. Open Web Application Security Project, 2023. <https://owasp.org/www-project-top-ten/>